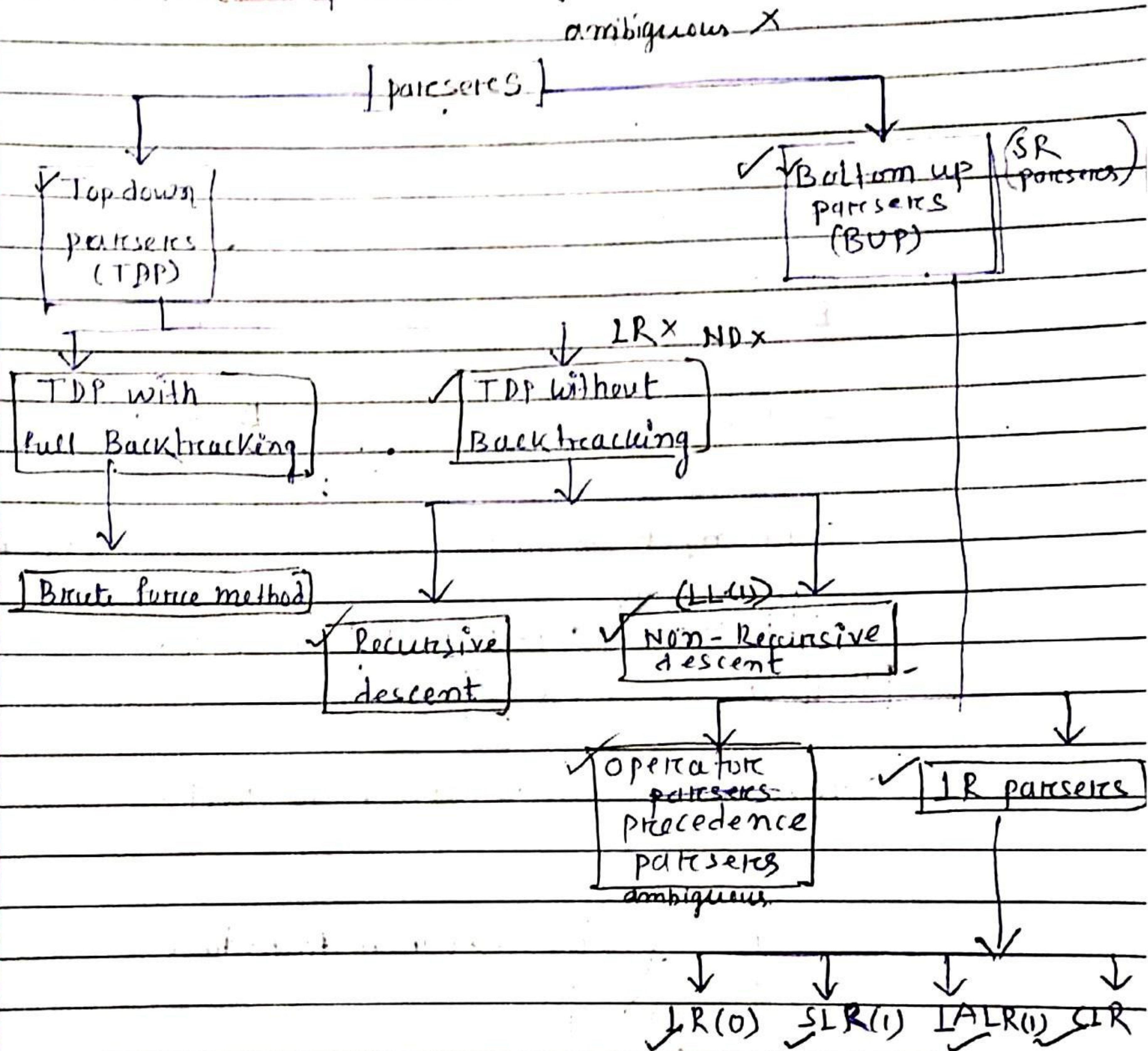


PARSERS

atlantis

17

Introduction of parsers :



→ Parsers is syntax analyser.

→ SR parsers means shift-reduce parsers.

→ TDP without backtracking not accept - Left recursive grammar and also Non-deterministic grammar.

→ Parsers not accept ambiguous grammar. only operator precedence parsers accept ambiguous grammar.

- Basic difference between Top-down parsers and Bottom up parsers:

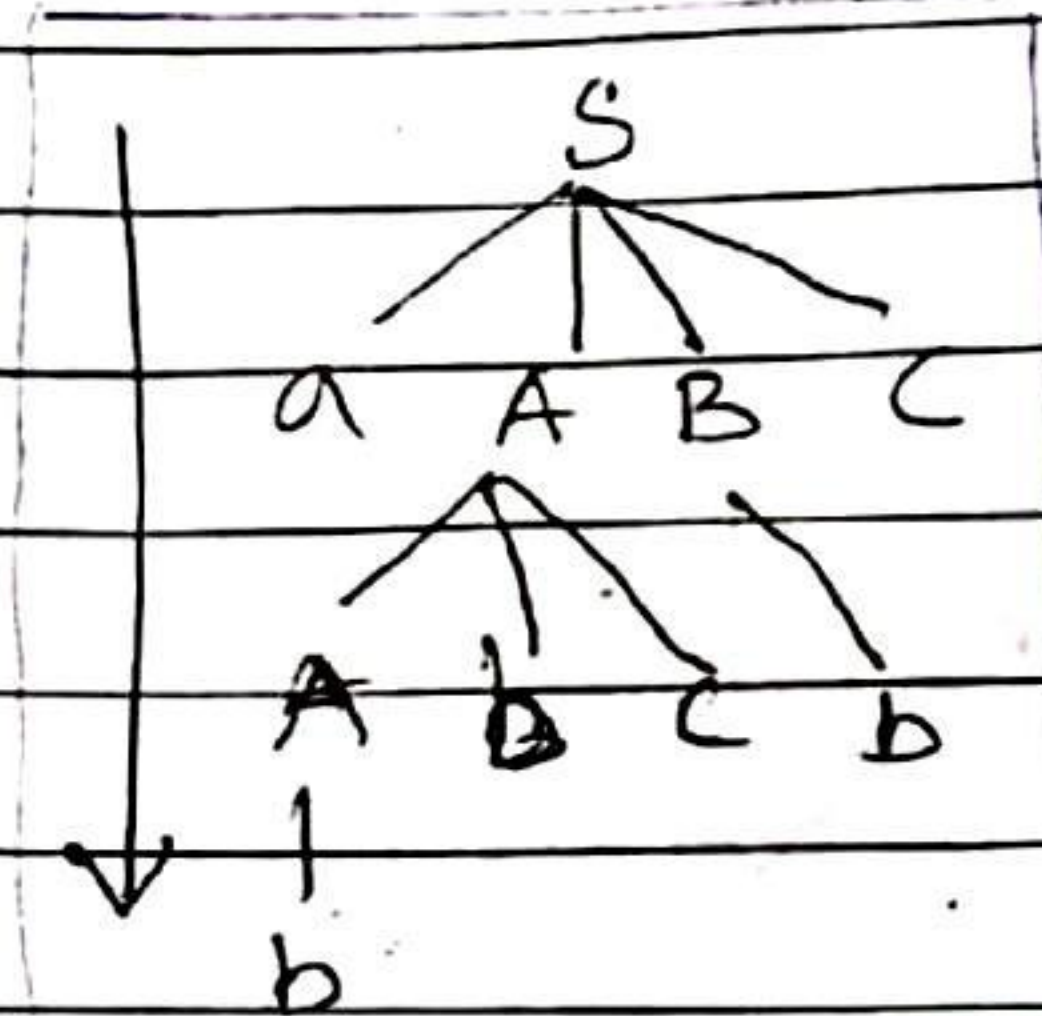
Top down parsers

$$S \rightarrow aABc$$

$$A \rightarrow Abc/b$$

$$B \rightarrow d$$

$$W = abbcd$$



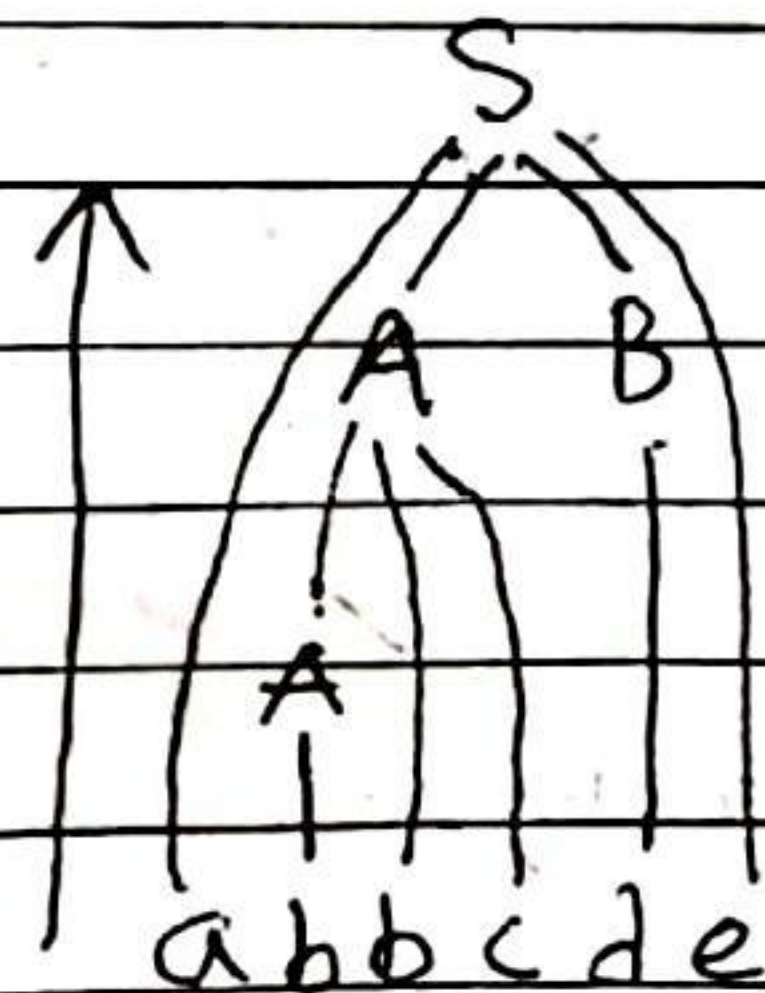
→ what to use.

→ Starting from the the start and going to the ~~per~~ terminal.

→ TDP also ~~also~~ follow Left most Derivation.

Bottom up parsers

→ take the terminals and go to root.



→ when to reduce

$$W = abbcd$$

$$S \Rightarrow a \textcircled{A} B e$$

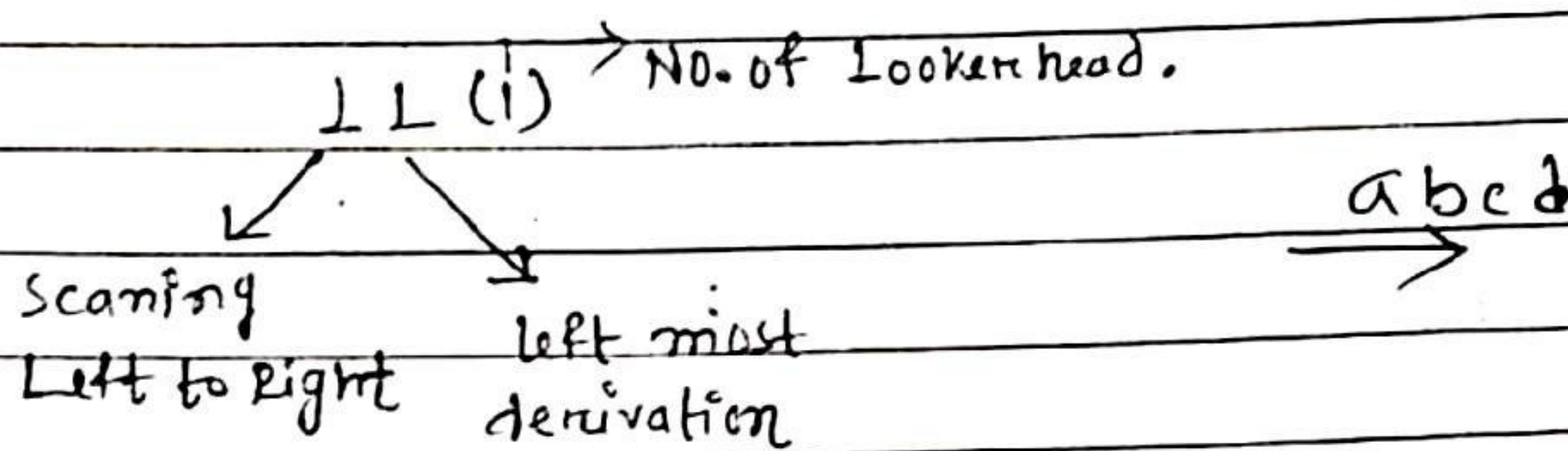
$$\Rightarrow a \textcircled{A} d e$$

$$\Rightarrow a \textcircled{Abc} d e$$

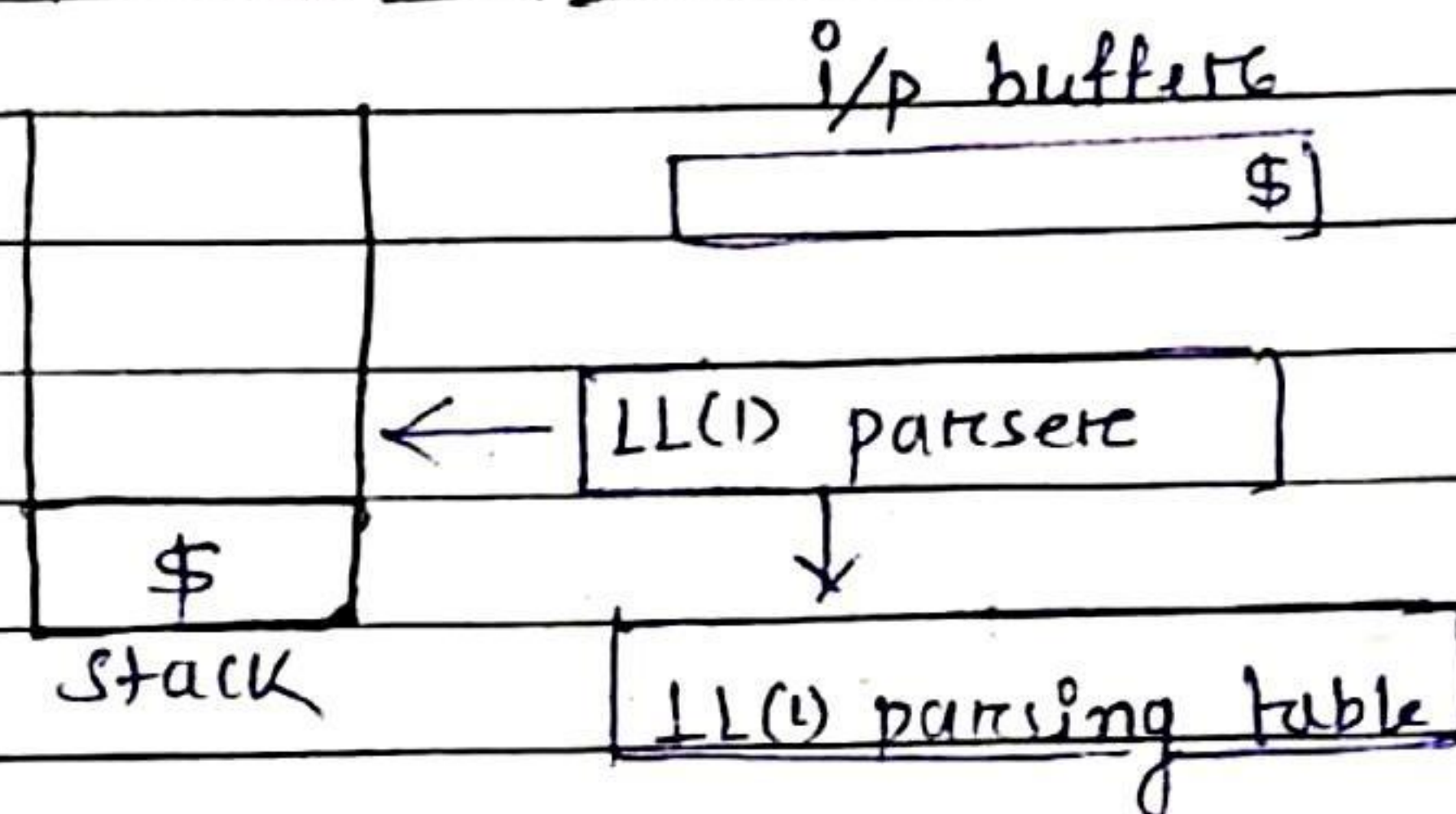
$$\Rightarrow a \textcircled{b} bcde$$

→ BUP follow Right most Derivation.

• LL(1) parsers:



• Components of parser =



$\rightarrow$ '\$' is to guess when stop.

• First()

• Follow()

First()

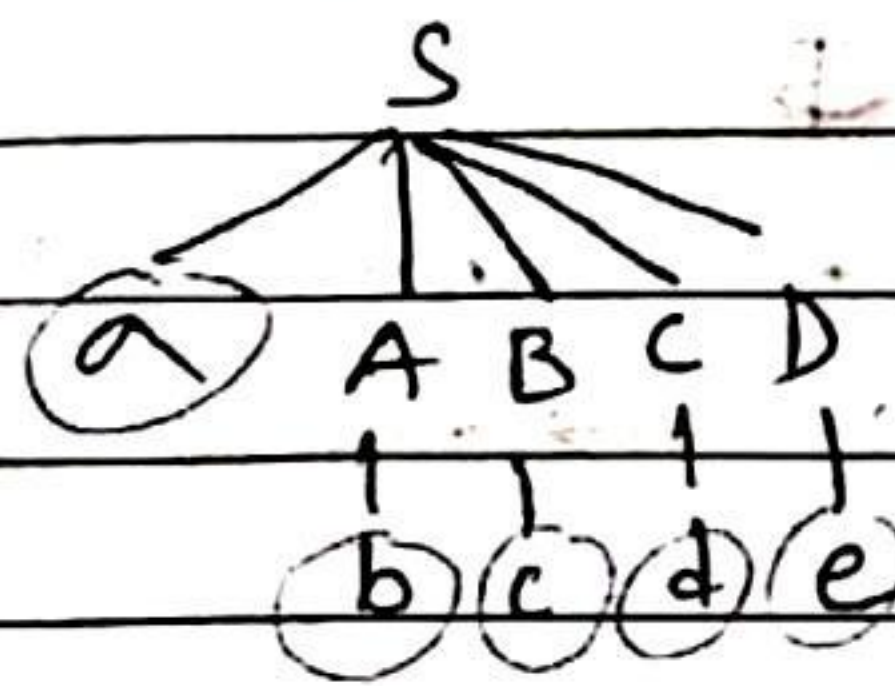
Ex: $S \rightarrow a A B C D$

$A \rightarrow b$

$B \rightarrow c$

$C \rightarrow d$

$D \rightarrow e$



here, first of S is a.

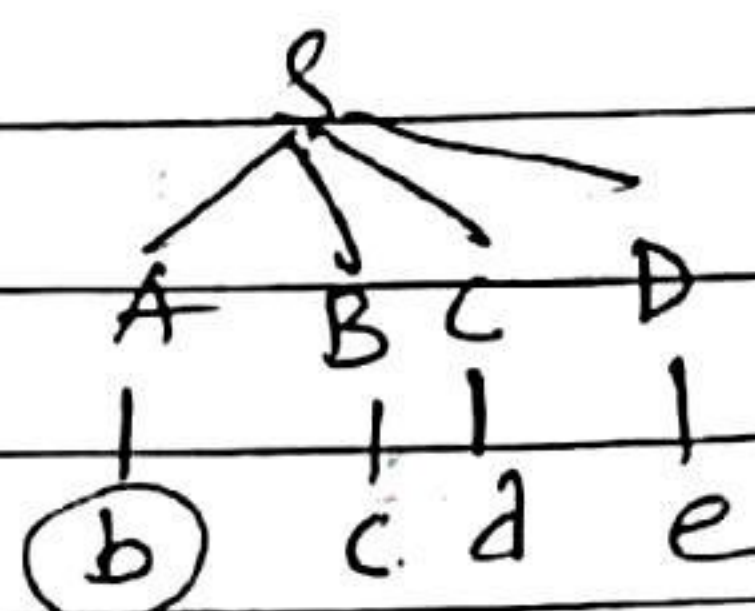
" " A " b

" " B " c

" " C " d

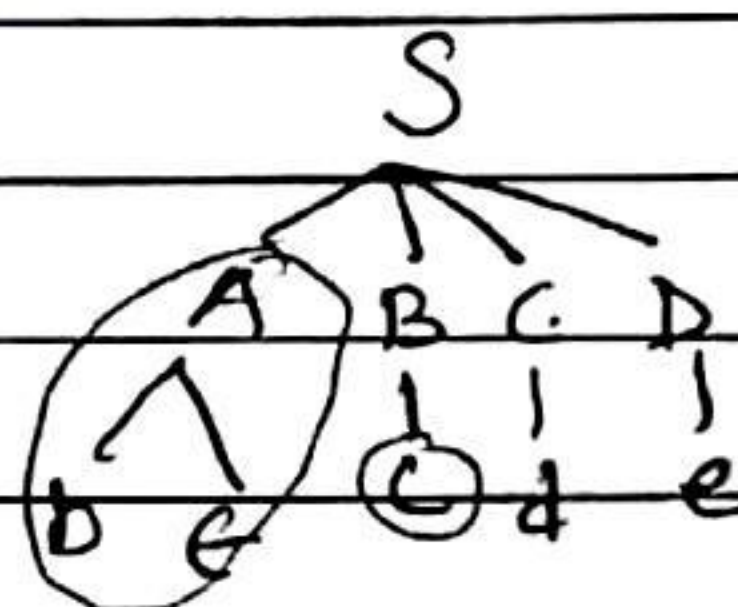
first of D " e

[Ex]-

 $S \rightarrow ABCD$ $A \rightarrow b$ $B \rightarrow c$ $C \rightarrow d$ $D \rightarrow e$ 

→ here first of S is 'b'.

[Ex]-

 $S \rightarrow \overset{x}{A}BCD$ $A \rightarrow b/\underline{e}$ $B \rightarrow c$ $C \rightarrow d$ $D \rightarrow e$ 

→ here in this case first of S is 'e'.

Follow:

What a terminal ^{which can} ~~that can~~ follow a variable in the process of derivation.

 $S \$$ $abcd \$$ $ABCD \$ \rightarrow$ here follow of D is '\$' $ABdD \$ \rightarrow$ follow of B is 'd'. $S \rightarrow ABCD$ $A \rightarrow b/\underline{e}$ $B \rightarrow c$ $C \rightarrow d$ $D \rightarrow e$

→ follow of D is follow of 'S'.

→ here in this case follow of S - { '\$' }

~~follow of A~~ follow of A is first of BCD = 'c'.

So, follow of A is { 'c' }.

Follow of B is $\{d\}$.

" "C is $\{e\}$.

Follow of D is $\{\$ \}$.

$A \rightarrow BC$

- Example - find first and follow in LL(1):

Ex-1

	first	follow
6 $S \rightarrow \overset{\epsilon}{A}BCDE$	$\{a, b, c\}$	$\{\$ \}$
1 $A \rightarrow a/\epsilon$	$\{a, \epsilon\}$	$\{b, c\}$
2 $B \rightarrow b/\epsilon$	$\{b, \epsilon\}$	$\{c\}$
3 $C \rightarrow c$	$\{c\}$	$\{a, e, \$ \}$
4 $D \rightarrow d/\epsilon$	$\{d, \epsilon\}$	$\{e, \$ \}$
5 $E \rightarrow e/\epsilon$	$\{e, \epsilon\}$	$\{\$ \}$

→ Follow of A is first of $BCDE = \{b, c\}$.

Ex-2

	first	follow
3 $S \rightarrow Bb/Cd$	$\{a, b, c, d\}$	$\{\$ \}$
1 $B \rightarrow aB/\epsilon$	$\{a, \epsilon\}$	$\{b\}$
2 $C \rightarrow cC/\epsilon$	$\{c, \epsilon\}$	$\{d\}$

Ex-3

	first	follow
5 $E \rightarrow TE'$	$\{id, \epsilon\}$	$\{\$, \}$
4 $E' \rightarrow +TE'/\epsilon$	$\{+, \epsilon\}$	$\{\$, \}$
3 $T \rightarrow FT$	$\{id, \epsilon\}$	$\{+, \$ \}$
2 $T' \rightarrow *FT'/\epsilon$	$\{*, \epsilon\}$	$\{+, \$, \}$
1 $F \rightarrow id/(\epsilon)$	$\{id, \epsilon\}$	$\{*, +, \$, \}$

Ex-4:

$$1 \quad S \rightarrow A(CB) / (C) b B / B a$$

$$3 \quad A \rightarrow d a / (B) C$$

$$2 \quad B \rightarrow g / \epsilon$$

$$1 \quad C \rightarrow h / \epsilon$$

first

{d, g, h, ϵ , b, a}{d, g, h, ϵ }{g, ϵ }{h, ϵ }

follow

{ ϵ }{h, g, ϵ }{ ϵ , a, h, g}{b, h, g, ϵ }Ex-5:

$$S \rightarrow a(A)Bb$$

$$A \rightarrow c / \epsilon$$

$$B \rightarrow d / \epsilon$$

first

{a}

{c, ϵ }{d, ϵ }

follow

{ ϵ }

{d, b}

{b}

Ex-6

$$S \rightarrow a(B)Dh$$

$$B \rightarrow c$$

$$C \rightarrow b(C) / \epsilon$$

$$D \rightarrow (E)F$$

$$E \rightarrow g / \epsilon$$

$$F \rightarrow f / \epsilon$$

first

{a}

{c}

{b, ϵ }{g, f, ϵ }{g, ϵ }{f, ϵ }

follow

{ ϵ }

{g, f, h}

{g, f, b}

{h}

{f, h}

{h}

$$A \rightarrow CAB$$

here follow of 'B' is = follow of 'A'.

$$A \rightarrow ABCD$$

here follow of B is first of 'C' and 'D'.

• Construction of LL(1) parsing table :

Example - 1

	first	follow
$E \rightarrow TE'$	$\{id, (\}$	$\{ \$,) \}$
$E' \rightarrow +TE' / \epsilon$	$\{ +, \epsilon \}$	$\{ \$,) \}$
$T \rightarrow FT'$	$\{ id, (\}$	$\{ +, \epsilon, \$,) \}$
$T' \rightarrow *FT' / \epsilon$	$\{ *, \epsilon \}$	$\{ +, \$,) \}$
$F \rightarrow id / (E)$	$\{ id, (\}$	$\{ *, +, \$,) \}$

→ ~~It is~~ LL(1) parser is a top down parser.

→ The main purpose of top down parser is when have two alternative for a variable, find out which production should choose.

Table

← terminals →

		id	+	*	(	)	\$
variables	E	$E \rightarrow TE'$			$E \rightarrow TE'$		
	E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
	T	$T \rightarrow FT'$			$T \rightarrow FT'$		
	T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
	F	$F \rightarrow id$				$F \rightarrow (F)$	

✓ → All the ' ϵ ' production have to place under follow of 'left hand side'.

✓ → Where should place the production is depends on the first of first of Right hand side.

→ To construct parse tree, this table is used.

- Use of parsing to construct a parse tree -

Example - 2 $((()))$

$S \rightarrow (s)/\epsilon$	first $\{(, \epsilon\}$	follow $\{ \$,) \}$
------------------------------	----------------------------	-------------------------

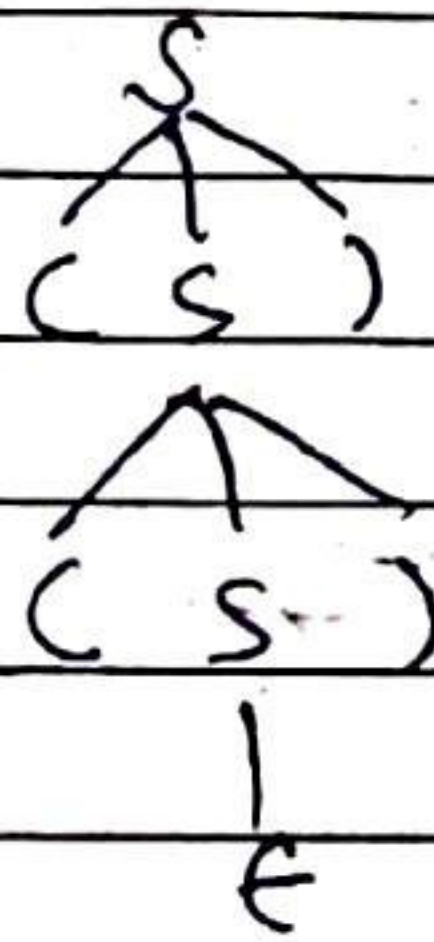
table:

	(	)	\$
S	$S \rightarrow (s)$	$S \rightarrow \epsilon$	$S \rightarrow \epsilon$

generate: $((())) \$$

$\$ | \$ | (|) | \$ | \$ | \$ | \$ | \$ | \$$

bottom of the stack



→ LL(1) parser algo start with '\$' in the bottom of the stack and '\$' in the end of the input.

when \$, \$ match then we can say the when we see \$ & \$ then we can say it is successful matching.

[Example-3]

	first	follow
$S \rightarrow AaAb / BbBa$	$\{ \epsilon, a, b \}$	$\{ \$ \}$
$A \rightarrow \epsilon$	$\{ \epsilon \}$	$\{ a, b \}$
$B \rightarrow \epsilon$	$\{ \epsilon \}$	$\{ b, a \}$

table =

	a	b	\$
S	$S \rightarrow AaAb$	$S \rightarrow BbBa$	
A	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	
B	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$	

cell of table

→ In every cell we got '1' production, so that ~~is~~ this grammar is LL(1).

→ Left recursion and Non-Deterministic grammars are not can not be used for LL(1).

$$A \rightarrow \alpha_1 / \alpha_2 / \alpha_3 / \alpha_4 / \dots / \alpha_n$$

A grammar has to be LL(1) when first of $\alpha_1, \alpha_2, \alpha_3, \dots, \alpha_n$ are mutually exclusive (means no common)

• Test that given Grammar is LL(1) or not:

	first	follow
① $S \rightarrow aSbS$ $/bSaS$ $/\epsilon$	$\{a, b, \epsilon\}$	$\{b, a, \$\}$

	a	b	\$
S	$S \rightarrow aSbS$ $S \rightarrow \epsilon$	$S \rightarrow bSaS$ $S \rightarrow \epsilon$	$S \rightarrow \epsilon$

→ one cell contain more than one production, so therefore this grammar is not LL(1).

	first	follow
② $S \rightarrow aA Bb$ (✓)	$\{a\}$	$\{\$, b\}$
$A \rightarrow c/\epsilon$	$\{c, \epsilon\}$	$\{a, b\}$
$B \rightarrow d/\epsilon$	$\{d, \epsilon\}$	$\{b\}$

	a	b	c	d	\$
S	$S \rightarrow aABb$				
A		$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	
B		$B \rightarrow \epsilon$		$B \rightarrow d$	

→ each cell contain only 1 production, therefore this given grammar is LL(1).

	first	follow
③ $S \rightarrow A/a$	$\{a\}$	$\{\$, \epsilon\}$
$A \rightarrow a$	$\{a\}$	$\{\$, \epsilon\}$

	a	\$
S	$S \rightarrow A, S \rightarrow a$	
A	$A \rightarrow a$	

→ cell contain more than 1 production, not LL(1).
→ also it is a ambiguous grammar.

④		first	follow
✓	$S \rightarrow aB / \epsilon$	$\{a, \epsilon\}$	$\{\$ \}$
	$B \rightarrow bC / \epsilon$	$\{b, \epsilon\}$	$\{\$ \}$
	$C \rightarrow cS / \epsilon$	$\{c, \epsilon\}$	$\{\$ \}$

table

	a	b	c	\$
S	$S \rightarrow aB$			$S \rightarrow \epsilon$
B		$B \rightarrow bC$		$B \rightarrow \epsilon$
C			$C \rightarrow cS$	$C \rightarrow \epsilon$

→ each cell contain single one production.
therefore LL(1).

⑤		first	follow
✓	$S \rightarrow AB$	$\{a, b, \epsilon\}$	$\{\$ \}$
	$A \rightarrow a / \epsilon$	$\{a, \epsilon\}$	$\{b, \$ \}$
	$B \rightarrow b / \epsilon$	$\{b, \epsilon\}$	$\{\$ \}$

Parser table =

	a	b	\$
S	$S \rightarrow AB$	$S \rightarrow AB$	$S \rightarrow AB$
A	$A \rightarrow a$	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$
B		$B \rightarrow b$	$B \rightarrow b$

→ each cell contain single one production,
therefore this grammar is LL(1).

⑥		first	follow
x	$S \rightarrow aSA / \epsilon$	$\{a, \epsilon\}$	$\{c, \$ \}$
	$A \rightarrow c / \epsilon$	$\{c, \epsilon\}$	$\{c, \$ \}$

	a	c	\$
S	$S \rightarrow aSA$	$S \rightarrow aSA \epsilon$	$S \rightarrow \epsilon$
A		$A \rightarrow c$ $A \rightarrow \epsilon$	$A \rightarrow \epsilon$

→ one cell contained more than '1' production, so therefore it is not LL(1).

⑦

	first	follow
$S \rightarrow A$	$\{a, b, c, d\}$	$\{\$ \}$
$A \rightarrow (B)b/cd$	$\{a, b, c, d\}$	$\{\$ \}$
$B \rightarrow a(B)/e$	$\{a, e\}$	$\{b\}$
$C \rightarrow cC/e$	$\{c, e\}$	$\{d\}$

table

	a	b	c	d	\$
S	$S \rightarrow A$	$S \rightarrow A$	$S \rightarrow A$	$S \rightarrow A$	
A	$A \rightarrow Bb$	$A \rightarrow Bb$	$A \rightarrow Bb$	$A \rightarrow Bb$	
B	$B \rightarrow aB$	$B \rightarrow e$			error
C			$C \rightarrow cC$	$C \rightarrow e$	

→ each cell contain only '1' production so it is LL(1).

⑧

	first	follow
$S \rightarrow aAa/e$	$\{a, e\}$	$\{\$, a\}$
$A \rightarrow abs/e$	$\{a, e\}$	$\{a\}$

	a	b	\$
S	$S \rightarrow aAa$		$S \rightarrow e$
A	$A \rightarrow abs$		

→ not LL(1).

⑨

$S \rightarrow (i e \& \& s$

⑨	first	follow.
$S \rightarrow iEtss' / a$	$\{i, a\}$	$\{\$, e\}$
$s' \rightarrow eS / e$	$\{e, e\}$	$\{\$, e\}$
$E \rightarrow b$	$\{b\}$	$\{t\}$

table

	i	t	a	b	e	\$
S	$S \rightarrow iEtss'$		$S \rightarrow a$			
s'					$s' \rightarrow eS$ $s' \rightarrow e$	$s' \rightarrow e$
E				$E \rightarrow b$		

→ It is not LL(1).

Recursive descent parser ==

Example

$E \rightarrow iE'$

$E' \rightarrow +iE' / e$

→ each ~~time~~ variable has a function.

$E()$

{

1. if ($l == i$)

2. { match('i');

3. $E'()$;

}

$E'()$

{

1. if ($l == +$)

{

2. match('i');

3. match('i');

4. $E'()$;

}

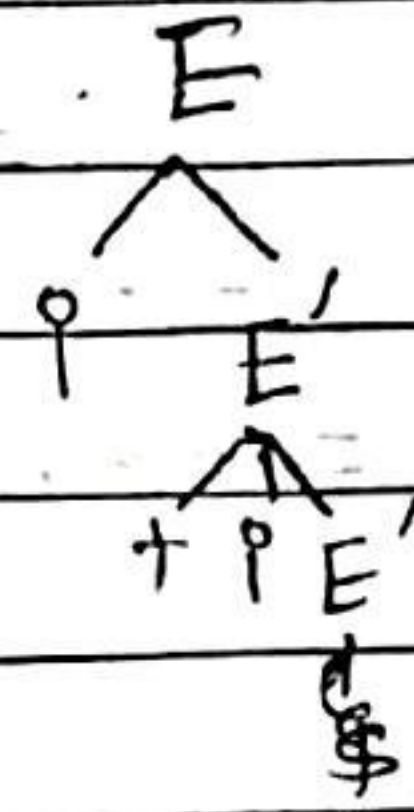
5. else

6. return;

4. }


```

<match (char t)
{
    if (l == t)
        l = getcharr();
    else
        print ("error");
}
    
```

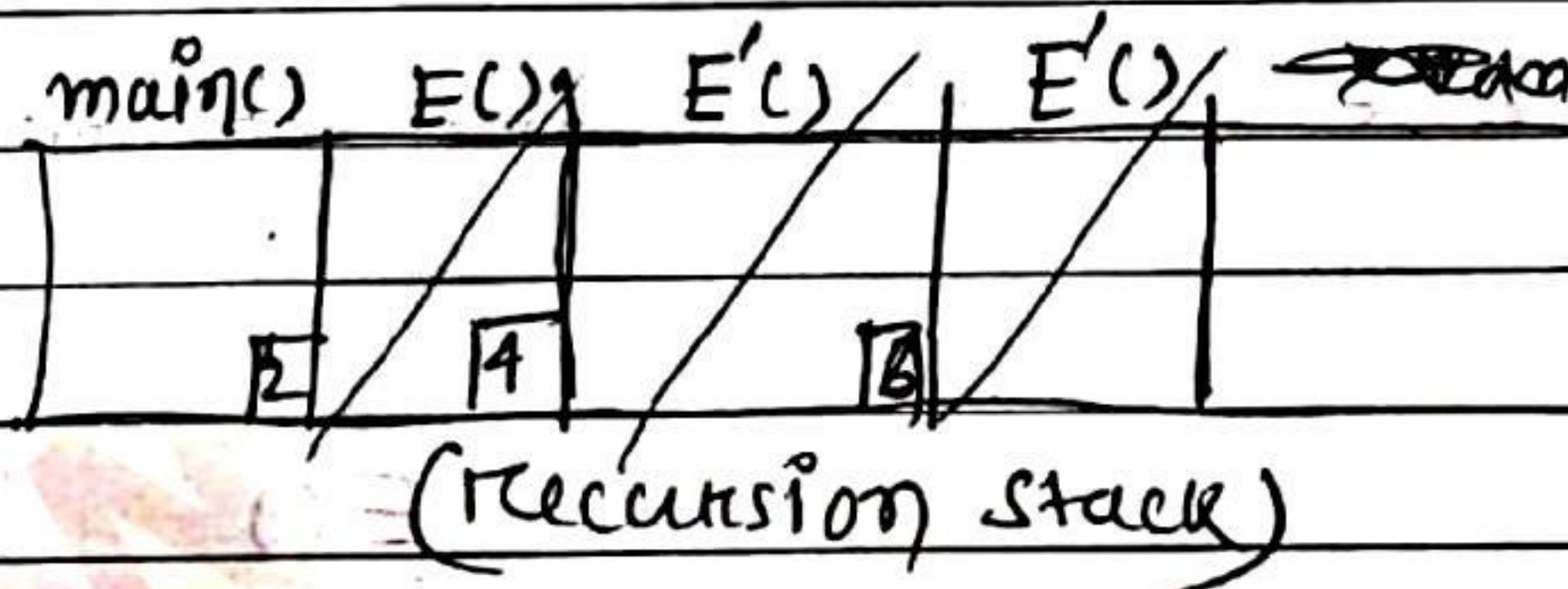


l = getcharr
(Incrementing)

```

main()
{
    1. E();
    2. if (l == '$')
    3. print ("parsing success");
}
    
```

according to ^{give} grammar ^{↓ ↓ ↓ ↓} i + i \$ generated.



→ write function for every variable, then call the function using the stack provided by operating system.

• Operator precedence parser: (Bottom up parsers)

• Operator grammar:-

✓ $E \rightarrow E + E / E * E / id$ $\rightarrow$ no two variable are adjacent.

X $E \rightarrow E A E / id$
 $A \rightarrow + / *$ $\Rightarrow E \rightarrow E + E / E * E / id$ ✓

Example: (given grammar are operators ^{precedence} grammar or not)

① $S \rightarrow S A S / a$

$A \rightarrow b S b / b$

$\Rightarrow$ this grammar is not operator precedence grammar, because two variable are not adjacent.

now going to convert this grammar into operator precedence grammar $\Leftarrow$

$S \rightarrow S b S b / S b S / a$

$A \rightarrow b S b / b$

$\rightarrow$ operator precedence g can parse ambiguous grammar.

• operation relation table:

$\rightarrow$ using this table we can ^{do} make parsing =

① example -

 $E \rightarrow E + E / E * E / id.$ $id > * > + > \$$

table =

	id	+	*	\$
id	-	>	>	>
+	<	>	<	>
*	<	>	>	>
\$	<	<	<	-

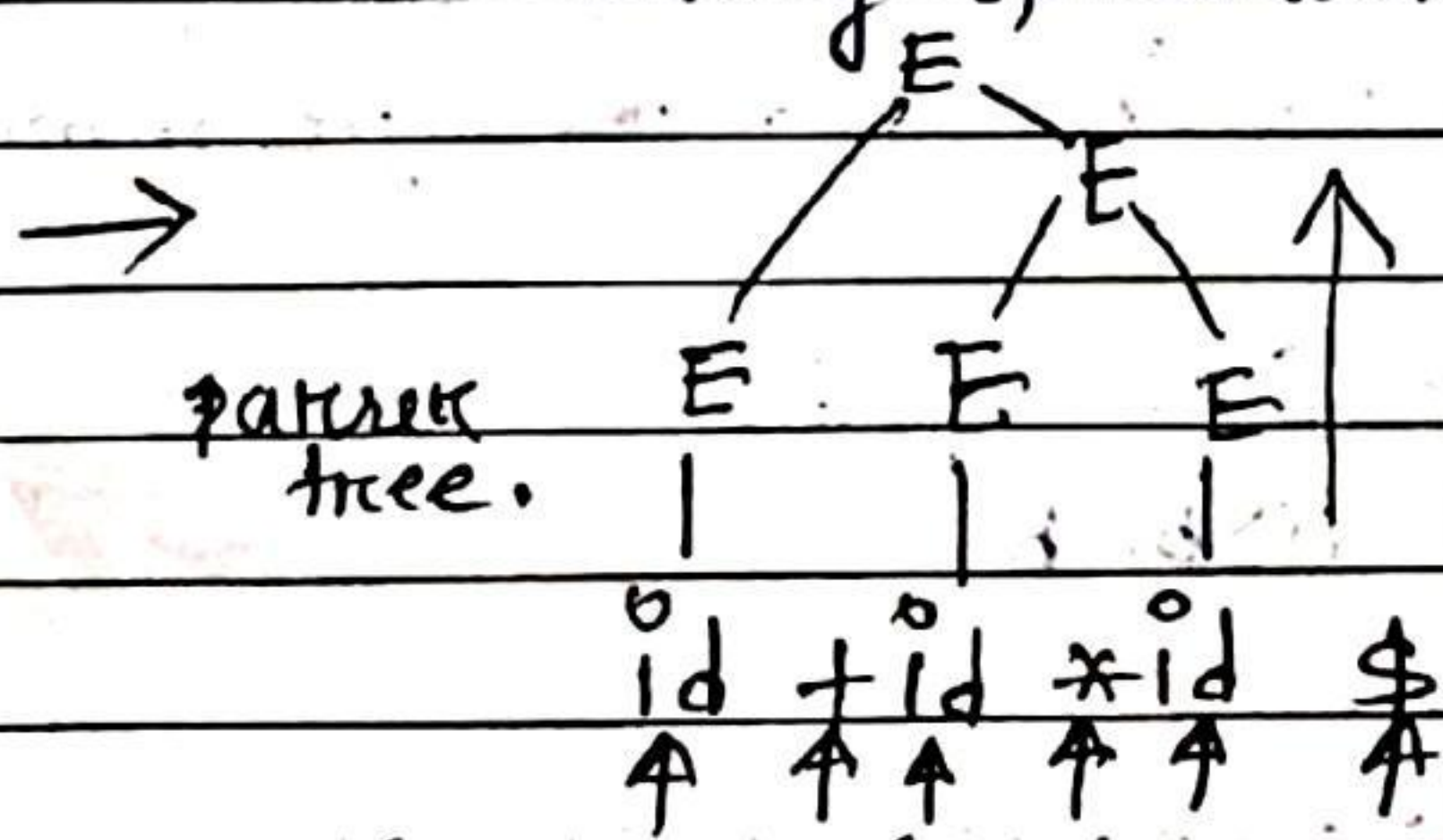
 $+ > +$ $* > *$

(operator precedence table)

(operator relation table)

Example ① parse $(id + id * id \$) \rightarrow i/p$

using operator precedence table.

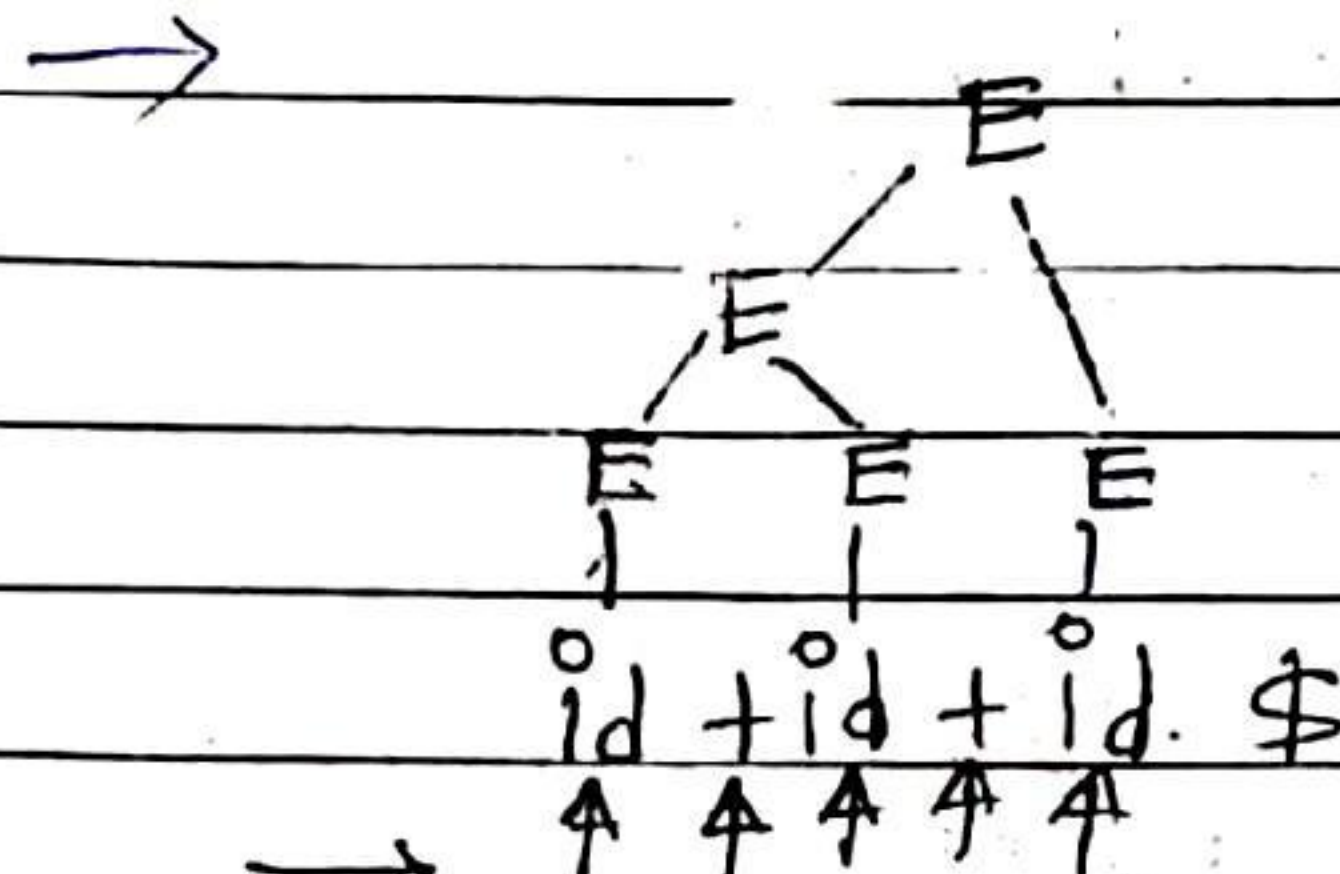
Stack: $[\$, id, +, id, *, id]$

- When top of the stack less than or equal to input then push i/p in stack.

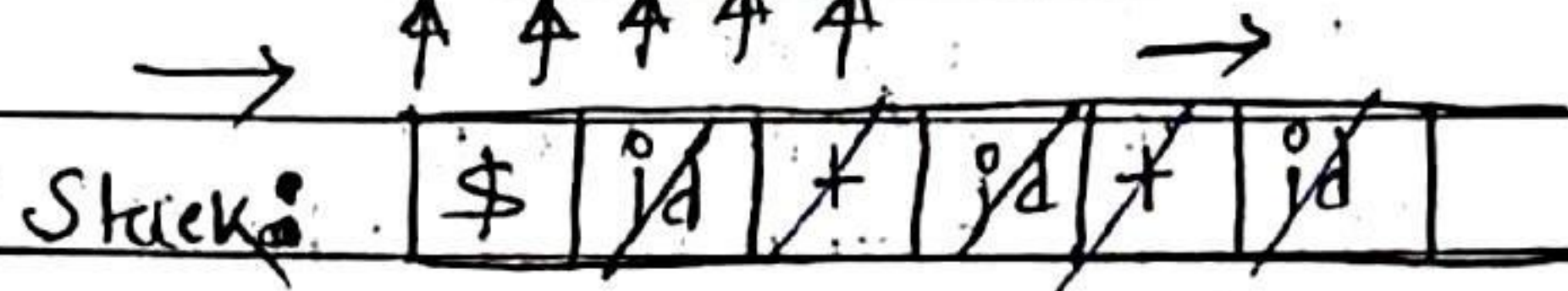
- When top of the stack are greater than to input then pop the top of the stack and terminate.

Example - 2

Construct parse tree of $id + id + id \$$ using operator precedence table.



→ left most '+' is greater than right most '+'.



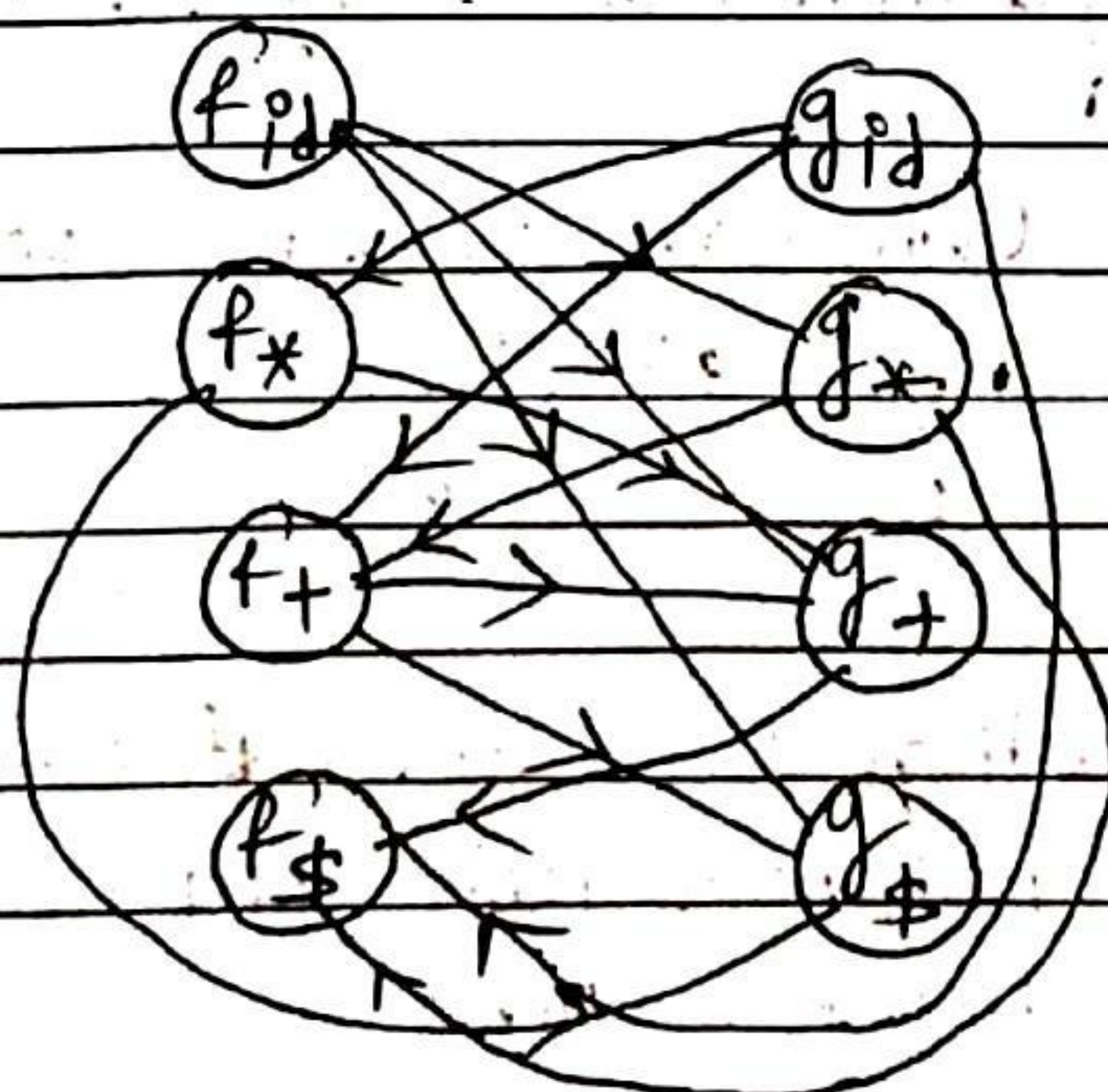
→ we will get one parse tree whether it is ambiguous.

• Disadvantage of operation relation table =

→ if we have '4' operators then size of the table are $4^2 = 16$.

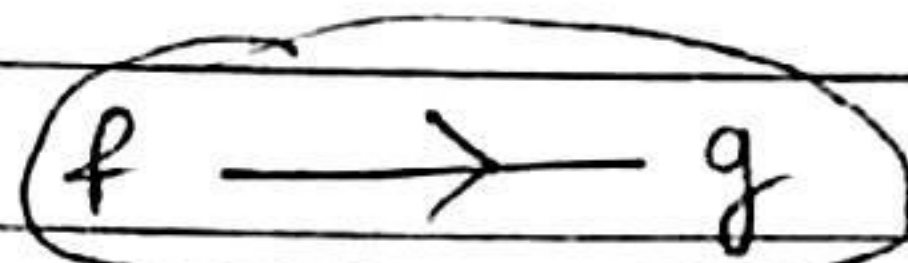
if we have 'n' operators then size of table are n^2 or $O(n^2)$.

→ To decrease the size of the table, we go for operator function table =

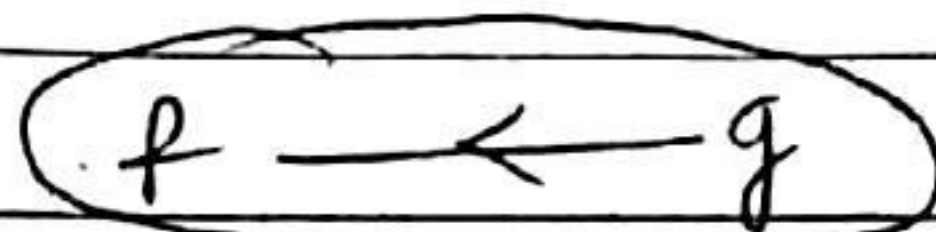


f	id	*	+	\$
id	-	>	>	>
*	<	>	>	>
+	<	<	>	>
\$	<	<	<	-

When greater than ($>$)



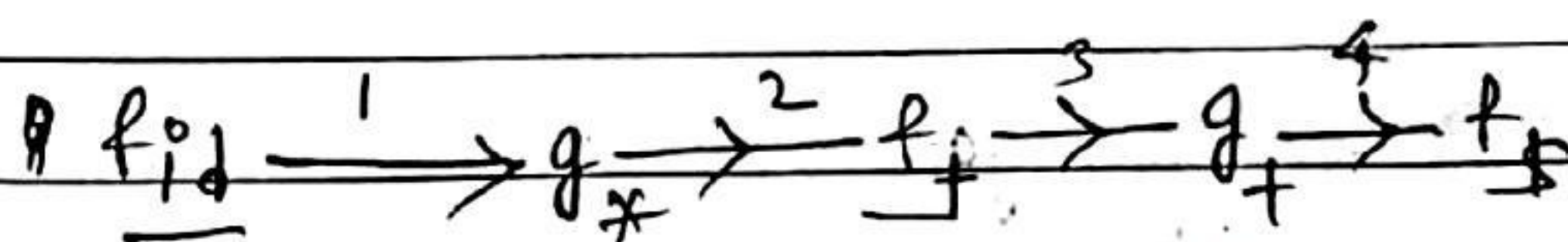
When less than ($<$)



any cycle

→ If in this graph, then stop we can't make function table.

→ find the longest path from the starting node.



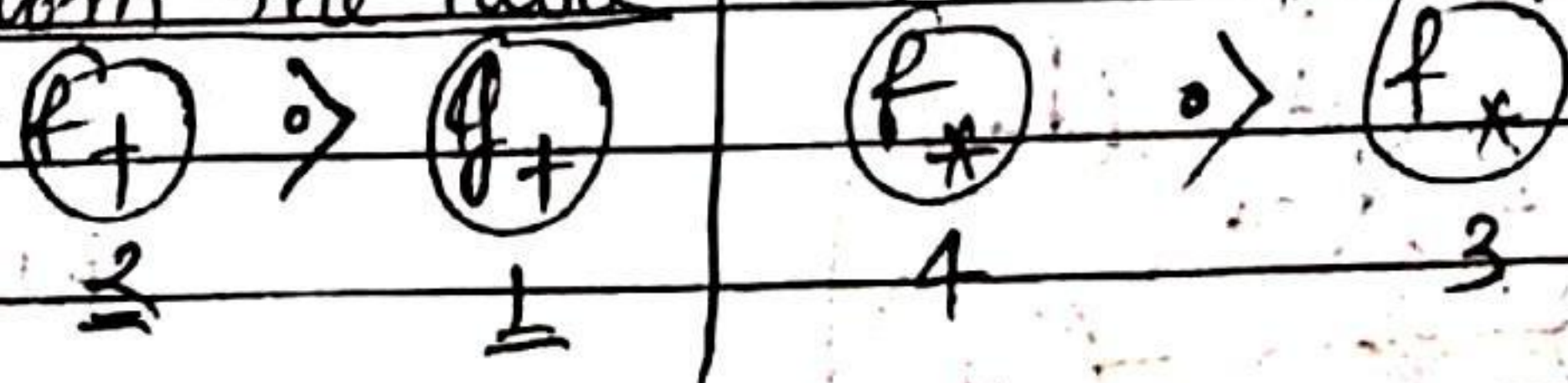
Function table

	id	+	*	#
f	4	2	4	0
g	5	1	3	0

→ This table contain same information (like relation table) but, the ~~size~~ of the size of table less.

with 'n' input operators, size of the table are $O(2^n)$

from the table = (we can compare operators)



→ Disadvantage of function table are, if when even blank entries in relation table, we get non-blank entries in function table.

example -

- $P \rightarrow (SR)/s \rightarrow$ not operator precedence gram.
- $R \rightarrow bSR/bS$ because two variable are adjacent.
- $S \rightarrow WbS/W$
- $W \rightarrow L*W/L$
- $L \rightarrow id$.

$\Rightarrow$ $P \rightarrow (SR)/s \rightarrow \textcircled{I}$
 $R \rightarrow bSR/bS \rightarrow \textcircled{II}$
 $S \rightarrow WbS/W$
 $W \rightarrow L*W/L$
 $L \rightarrow id$

$P \rightarrow$ paragraph.

$S \rightarrow$ sentence

$R \rightarrow$ recursive

$b \rightarrow$ Blank

$W \rightarrow$ word

$L \rightarrow$ letter.

$id \rightarrow$ Identifier.

conversion into operator precedence grammar.

$P \rightarrow Sb(SR)/SbS/S$

$\rightarrow P \rightarrow sbP/sbS/s$

$S \rightarrow WbS/W \rightarrow$ 'b' is right associative.

$W \rightarrow L*W/L \rightarrow$ so right most 'b' will get

$L \rightarrow id$. $\rightarrow$ '*' also right associative. higher precedence compare to left 'b'.

Operator relation table -

	id	*	b	\$
id	—	$\cdot >$	$\cdot >$	$\cdot >$
*	$< \cdot$	$< \cdot$	$\cdot >$	$\cdot >$
b	$< \cdot$	$< \cdot$	$< \cdot$	$\cdot >$
\$	$< \cdot$	$< \cdot$	$< \cdot$	—

$\rightarrow$ if grammar are not ambiguous then we can make table from given grammar.

Q. table =

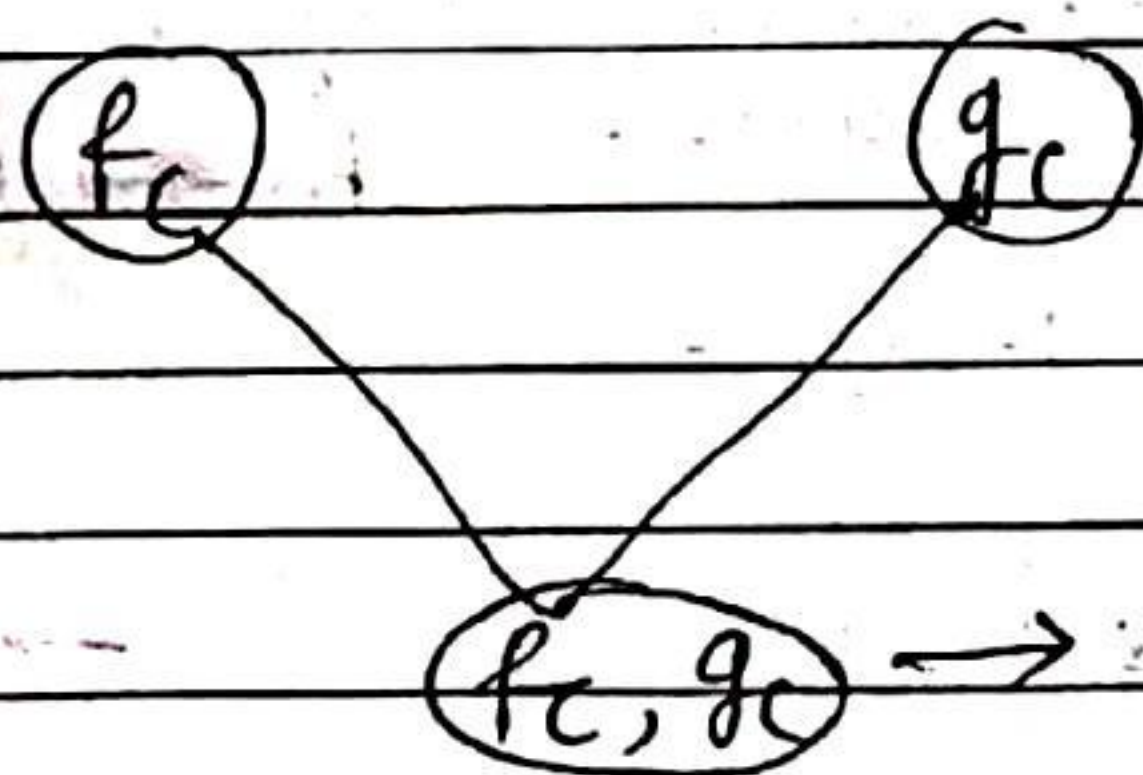
	a	(	)	,	\$
a			>	>	>
(	<	<	=	<	
)			>	>	>
,	<	<	>	>	
\$	<	<			

(operator relation table)

	a	(	)	,	\$
f	2	0	2	2	0
g	3	3	0	1	0

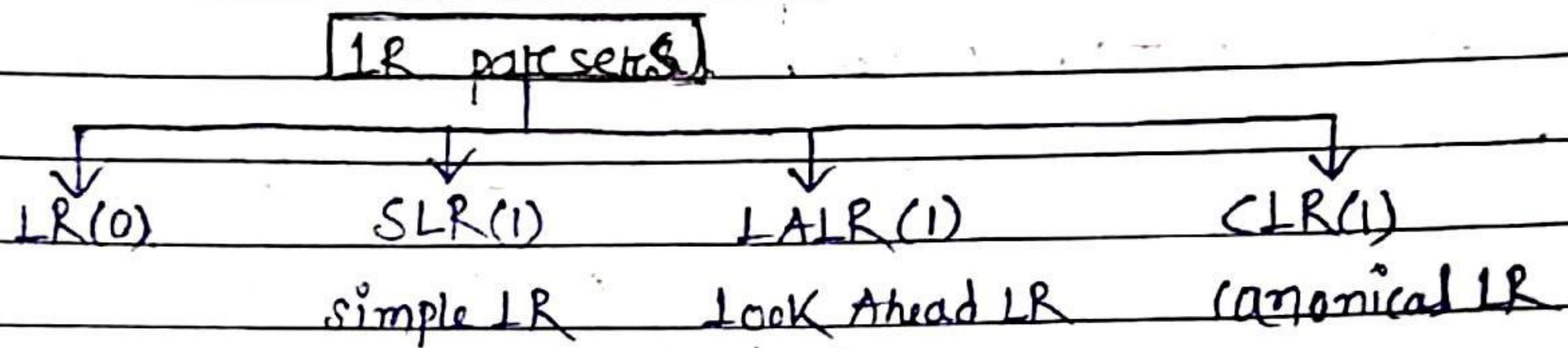
(operator function table)

$$f_c = g_c$$

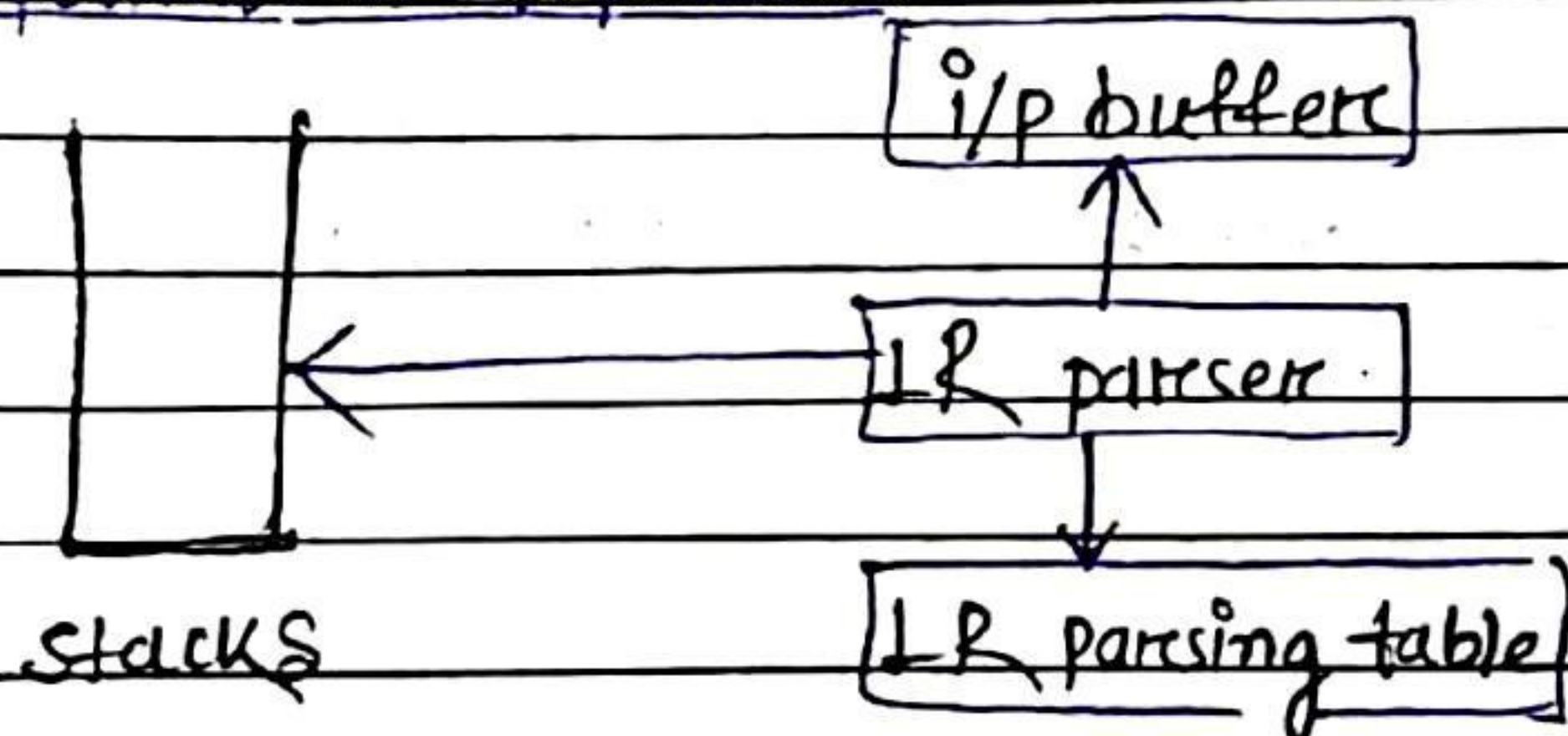


→ both w same value.

- LR parsers: (Bottom up parser)



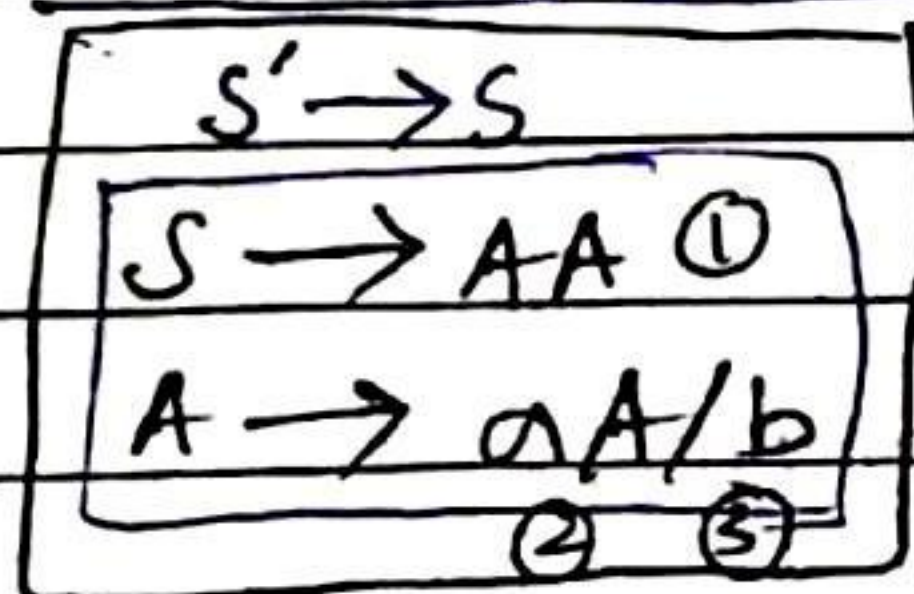
Components of LR parsers =



→ LR(0) items is used to construct the LR(0) and SLR(1) parsers.

→ LR(1) items is used to construct the LALR(1) and CLR(1) parsers.

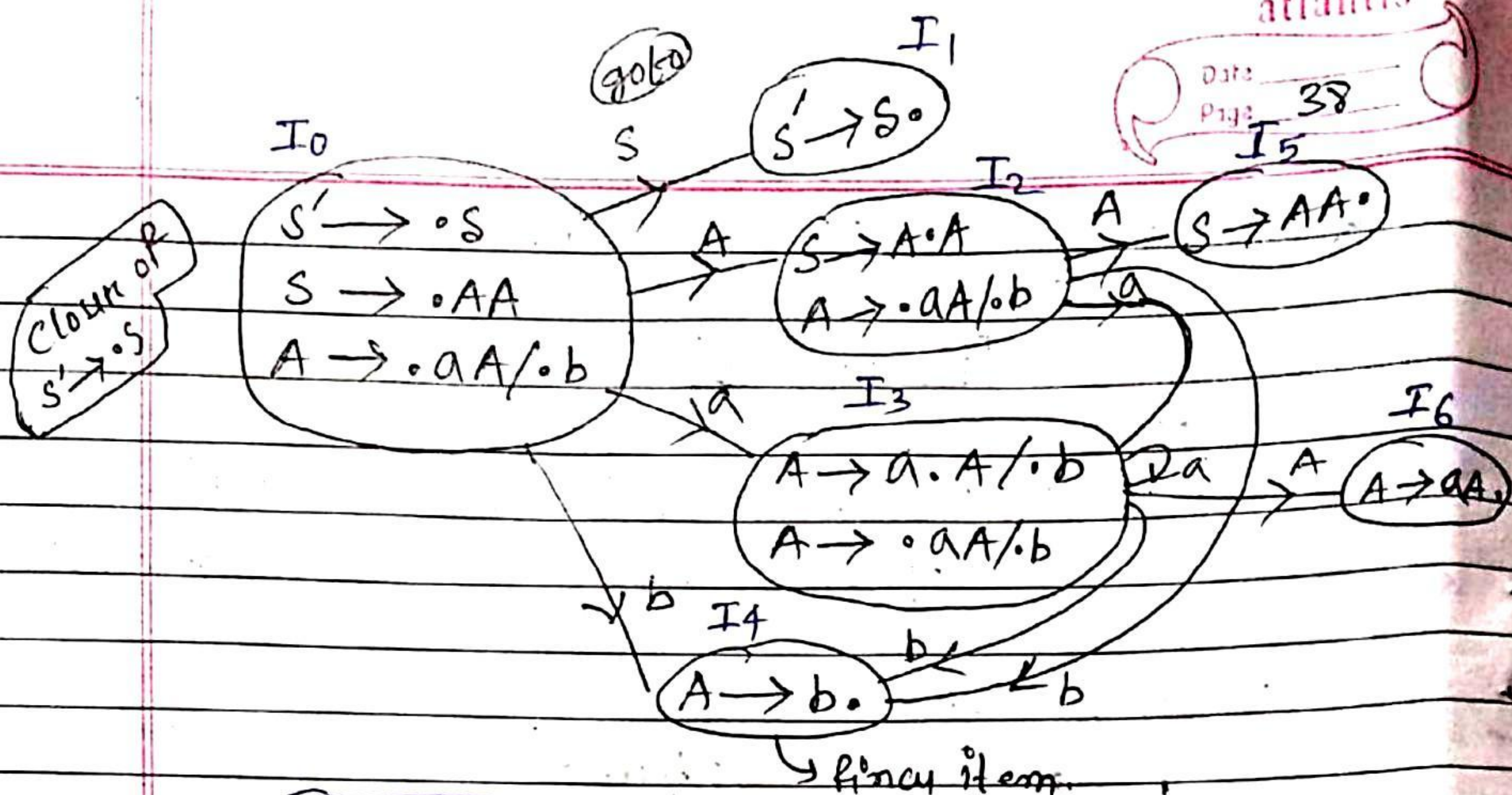
Construct LR(0) parsing table =



→ add one more production with given grammar.

→ closure: When a dot is left of a variable we have to add all the production dot in the left side of

$S' \rightarrow \cdot S$
 $S \rightarrow \cdot AA$
 $A \rightarrow \cdot aA/b$



Canonical collection of LR(0) items

Steps —

- (i) Take the grammar add a new production.
- (ii) and start with first ~~first~~ production by dot of the set left of S .
- (iii) then ~~all~~ applied closure.
- (iv) then find out transition (goto move) and again apply closure.

→ some states contain final items.

(any item dot at the end is called final item)

• LR(0) parsing table for canonical collection of LR(0) items —

→ next page —

table	terminals			variable	
	action part			goto part	
	a	b	\$	A	S
0	s ₃	s ₄		2	1
1			accept	-	
2	s ₃	s ₄	☐	5	
3	s ₃	s ₄	☐	6	
4	r ₃	(r ₃)	r ₃		
5	r ₁	r ₁	r ₁		
6	r ₂	r ₂	r ₂		

$\{r_3 \rightarrow \text{reduced to blank entries in table corresponding to the error.}$
 $\{s_3 \rightarrow \text{shift}$

Example: (use LR(0) parsing table to parsing)

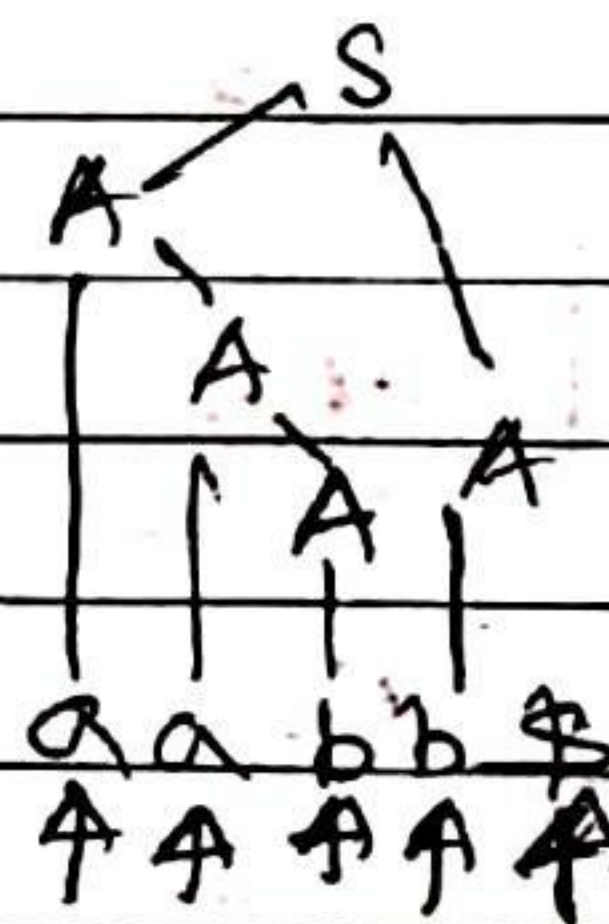
$s' \rightarrow S$

$S \rightarrow AA$ ①

$A \rightarrow aA/b$
② ③

generate aabb\$ using table.

→



~~aabb~~

stack

0	a	3	a	3	b	A	A	b	A	b	A	2	b	A	5	S	
---	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--------------	--

(accepted).

$A \rightarrow b$ (reduced)

$A \rightarrow aA$ (reduced)

Eg reduced

$A \rightarrow aA$ (reduced)

$A \rightarrow b$ "

$A \rightarrow AA$ "

• SLR(1) parsing

→ The main difference between LR(0) and SLR(1) is Reduce moves.

→ SLR(1) more powerful than LR(0).

$S' \rightarrow S$

$\left[\begin{array}{l} S \rightarrow AA \text{ (1)} \\ A \rightarrow aA/b \text{ (2) (3)} \end{array} \right] \rightarrow \text{grammar.}$

→ SLR(1) has less number of reduce moves compare to LR(0).
(Detect the error faster)

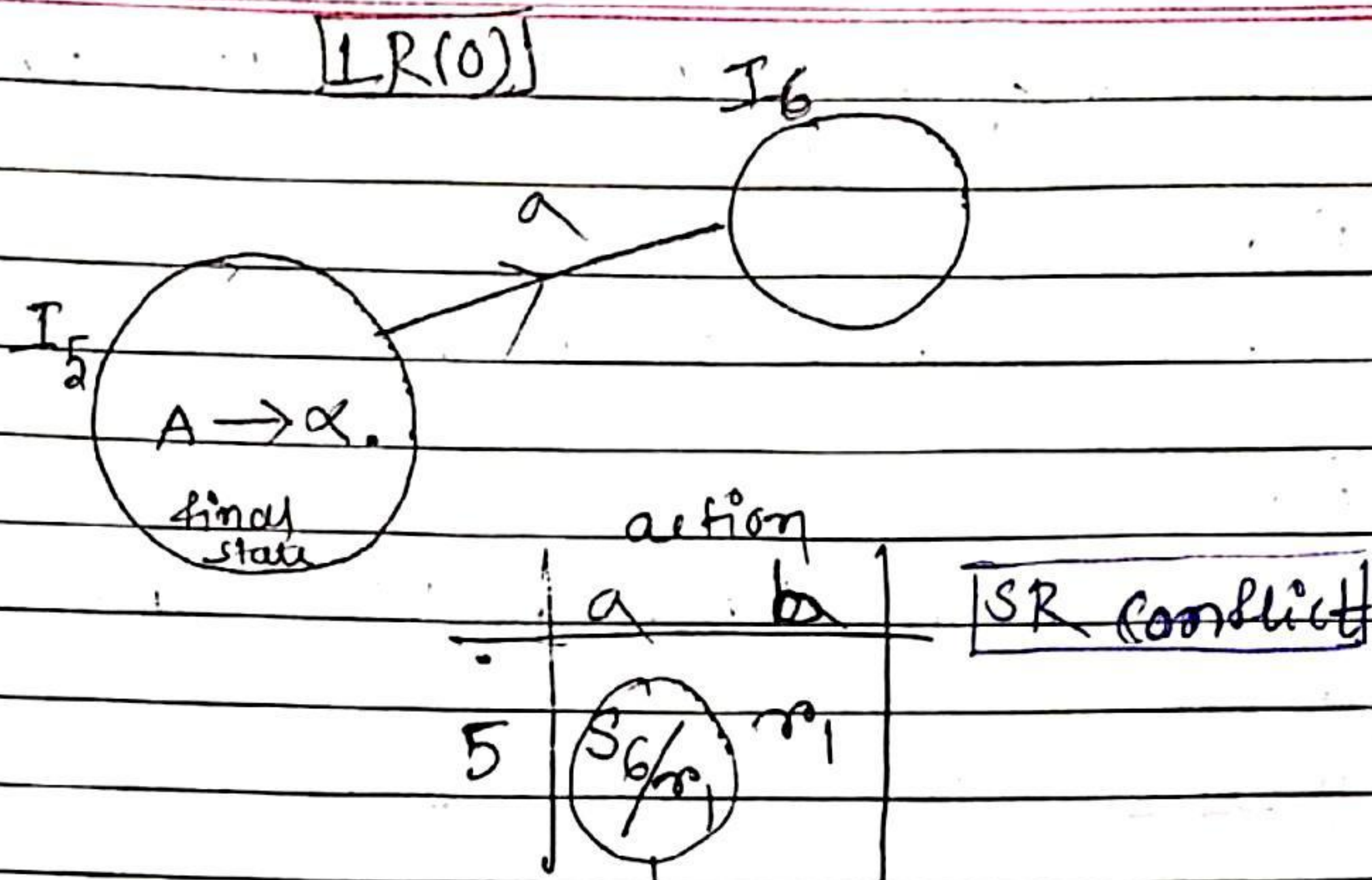
~~for~~ SLR(1) parsing table: (Make the way same like LR(0))

	<u>action</u>			<u>goto</u>	
	a	b	\$	A	S
0	S ₃	S ₄		2	1
1			accept		
2	S ₃	S ₄		5	
3	S ₃	S ₄		6	
4	r ₃	r ₃	r ₃		
5			r ₁		
6	r ₂	r ₂	r ₂		

→ just change in reduce move.

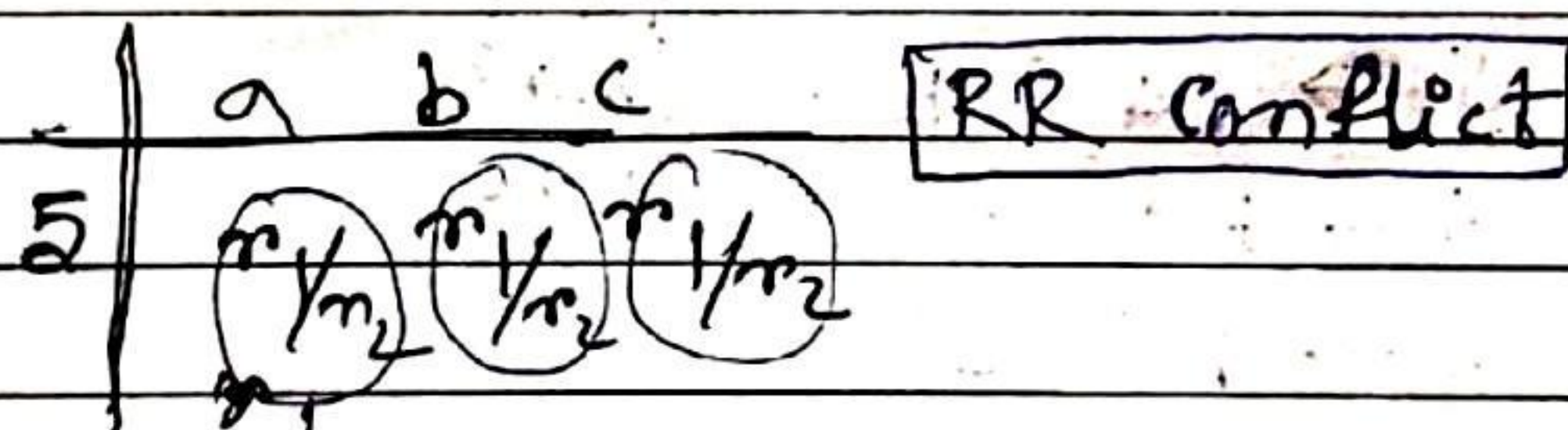
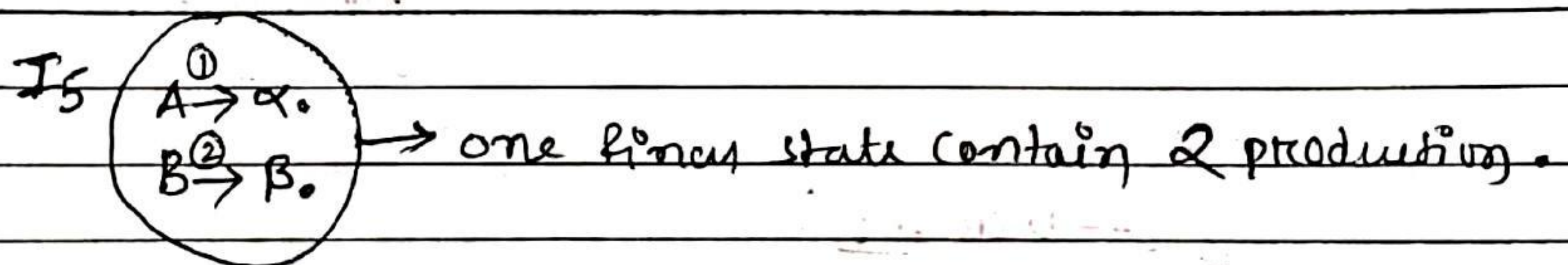
take final state = (find follow of right L.H.S)

$S' \rightarrow S \cdot$ I₁ $S \rightarrow AA \cdot$ I₅ $A \rightarrow aA \cdot$ I₆ $A \rightarrow b \cdot$ I₄
 $\$ \text{ (end)} \quad (a, b, \$) \quad (a, b, \$)$



→ called shift reduce conflict.

Not all grammars are not suitable for LR(0) parsing.



→ called RR conflict.

✓ → a grammar can't be LR(0) when it is RR conflict and SR conflict.

→ A grammar can't be LR(1) when it is RR conflict.

• chances of getting LR, SR for SLR(1) parsing =

(SR)

When $\text{follow}(A) = \{a\}$.

(LR)

When $\text{follow}(A) \cap \text{follow}(B) \neq \emptyset$.

Find out

• Given grammar also LR(0)? ~~no~~, SLR(1) or not?

①

$S \rightarrow dA/aB$
 $A \rightarrow bA/c$
 $B \rightarrow bB/c$

~~the~~ ~~the~~ given grammar also - (i) LL(1) or not.

✓ (ii) LR(0) or not.

✓ (iii) SLR(1) or not.

→

✓ LL(1) or not:

	first	follow
$S \rightarrow dA/aB$	$\{d, a\}$	$\{\$ \}$
$A \rightarrow bA/c$	$\{b, c\}$	$\{\$ \}$
$B \rightarrow bB/c$	$\{b, c\}$	$\{\$ \}$

	a	b	c	d	\$
S	$S \rightarrow aB$			$S \rightarrow dA$	
A		$A \rightarrow bA$	$A \rightarrow c$		
B		$B \rightarrow bB$	$B \rightarrow c$		

→ each cell containing only one '1' production so this grammar is LL(1).

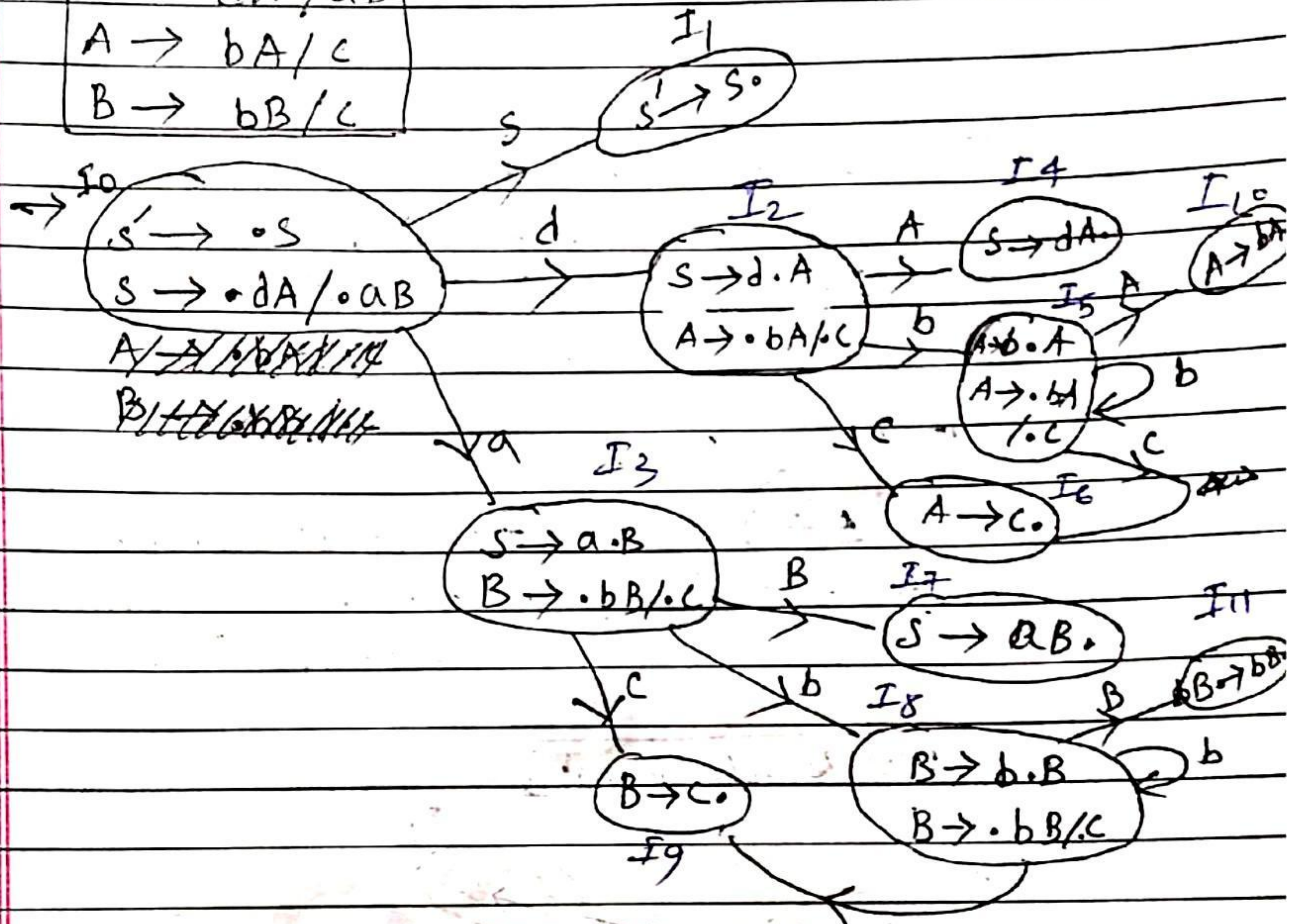
LR(0), SLR(1) or not

$S' \rightarrow S$ (added)

$S \rightarrow dA / aB$

$A \rightarrow bA / c$

$B \rightarrow bB / c$



→ here, no SR (shift reduce) conflict.

(Because each final state doesn't have any shift)

→ no RR conflict (Because each final state contains only one ~~RR~~ ~~product~~ product on)

So, this grammar is LR(0).

→ When a grammar is LR(0), then this grammar also SLR(1).

②

$S \rightarrow A/a$
 $A \rightarrow a$

check, given grammar are - LL(1) or not
- LR(0) " X
- SLR(1) " X

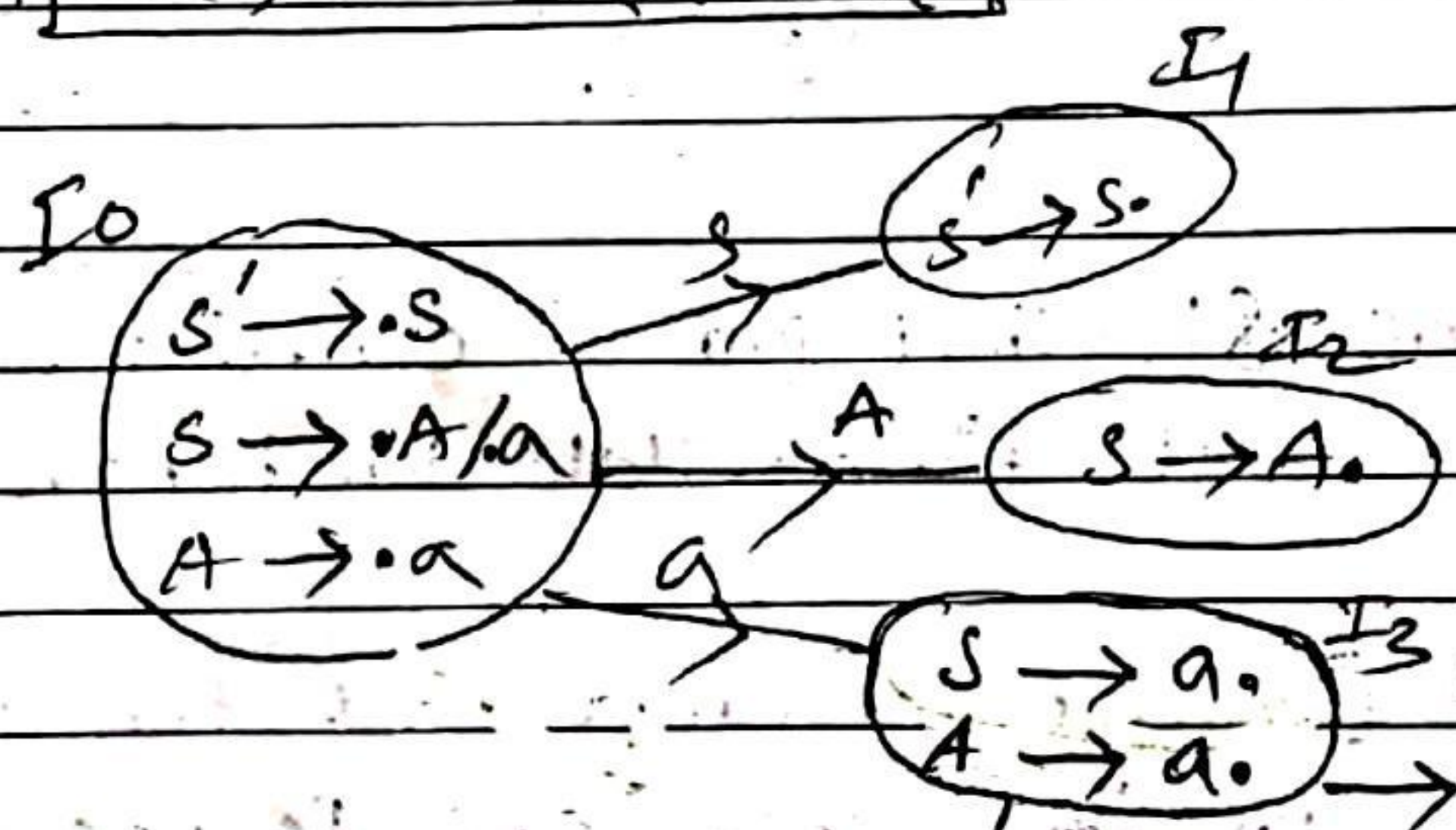
→ ~~SLR(1)~~ LL(1) or not

	first	follow
$S \rightarrow A/a$	$\{a\}$	$\{\$ \}$
$A \rightarrow a$	$\{a\}$	$\{\$ \}$

	a	\$
S	$S \rightarrow A/a$	
A	$A \rightarrow a$	

→ not ~~is~~ switabale for LL(1) parser because cell contain '2' production.

LR(0) & SLR or not



one final state contain '2' reduce move so it is ~~is~~ not switabale for LR(0) parsers.

follow of S and A is common '\$', so this grammar are not switabale for SLR(0) parsers.

③ Check given grammar are suitable for ~~LL(1)~~ or not

$$S \rightarrow (L) / a$$

$$L \rightarrow L, S / S$$

① LR(0) "

② SLR(1) "

LL(1) or not

$S \rightarrow (L) / a$ first $\{ (, a \}$

$L \rightarrow L, S / S$ first $\{ (, a \}$

follow $\{ \epsilon \}$

$\{ \epsilon \}$

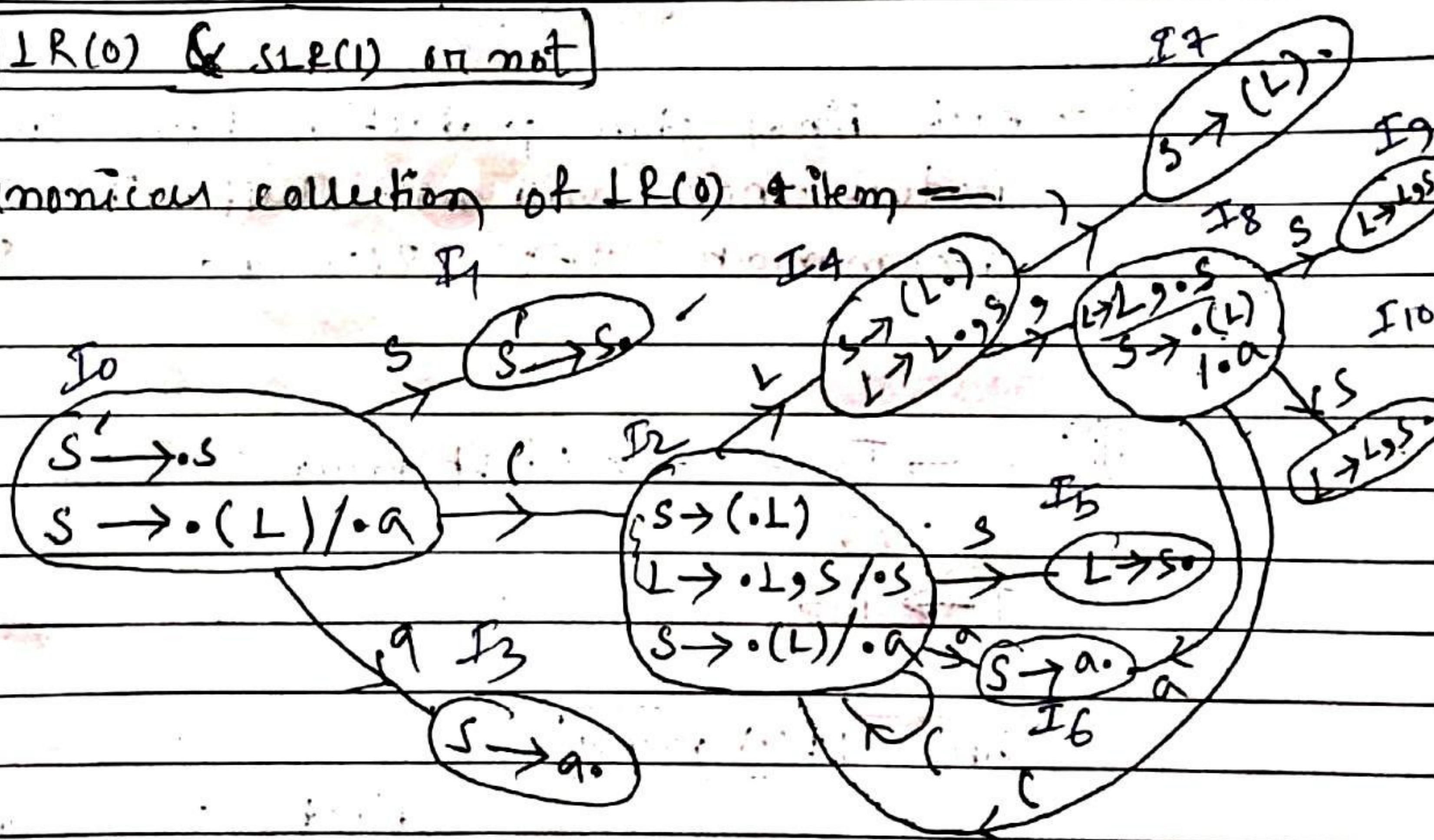
$\{ , ,) \}$

	(	,	a	)	\$
S	$S \rightarrow (L)$		$S \rightarrow a$		
L	$L \rightarrow L, S$		$L \rightarrow S$		

This grammar are left recursive grammar, so any left recursive can't be LL(1).

LR(0) & SLR(1) or not

canonical collection of LR(0) item =



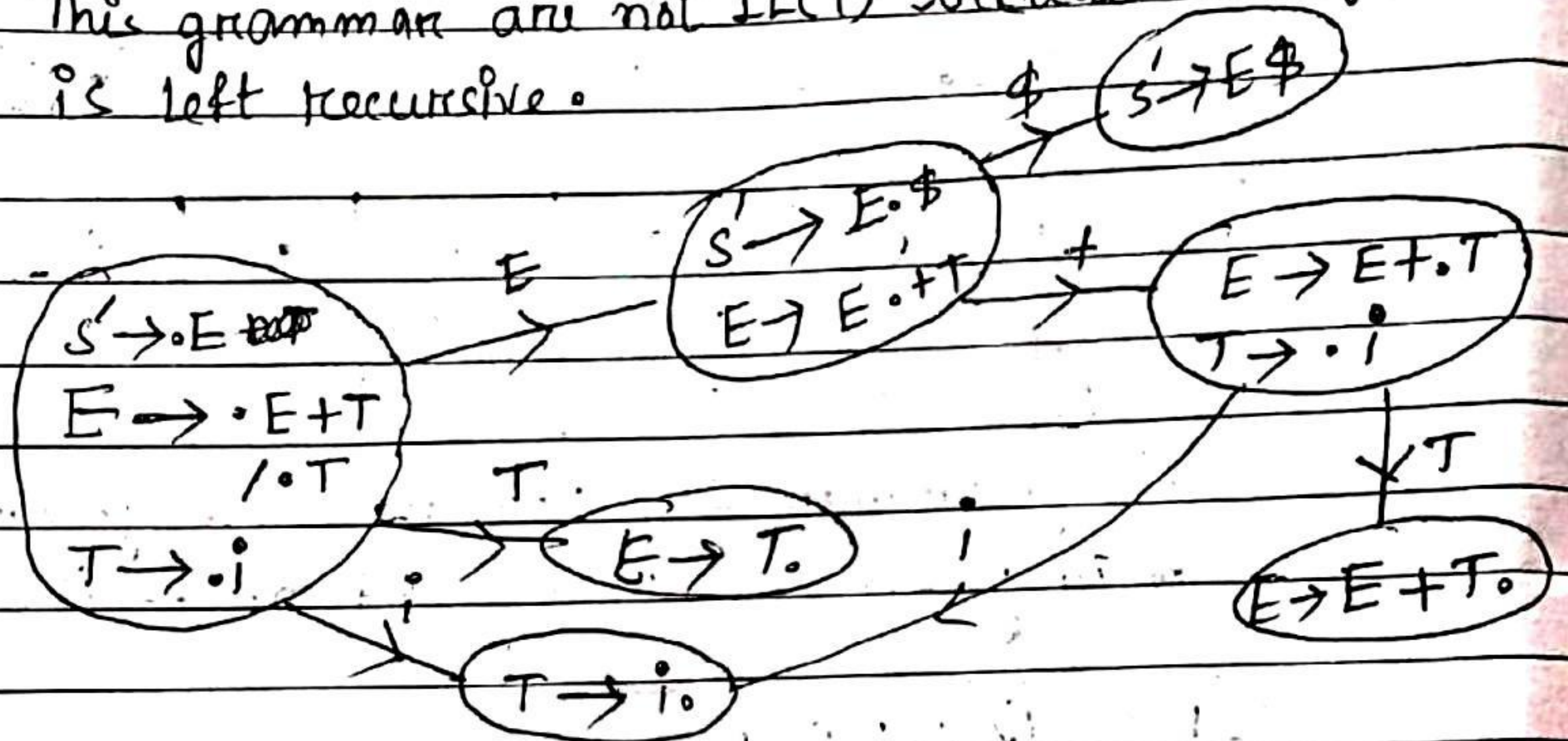
→ each times state contain only one production and doesn't have any shift. so, this grammar is suitable for LR(0).

→ SLR(1) also (because any are suitable for LR(0) and suitable for SLR(1)).

④ Check given grammars are suitable for LL(1), LR(0), SLR(1) or not.

i) $E \rightarrow E+T$
 $T \rightarrow i$
 $\rightarrow$ left recursive grammar.

$\Rightarrow$ This grammar are not LL(1) because this grammar is left recursive.



$\Rightarrow$ each final state production have single reduce move and not shift. Therefore given grammar are LR(0) and also SLR(1).

ii) $E \rightarrow T+E$ ① $\rightarrow$ Right recursive grammar.
 $T \rightarrow i$ ②

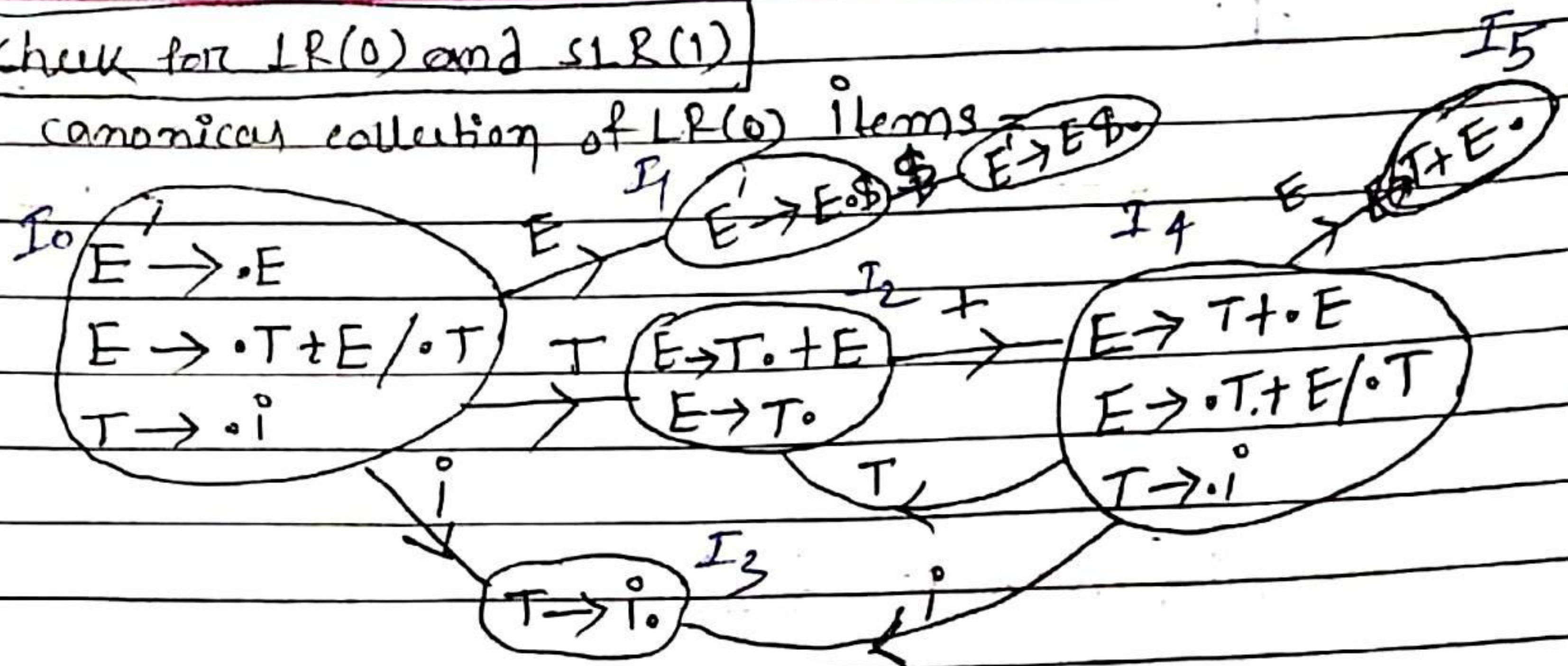
check LL(1) or not.

		first	follow
$E \rightarrow T+E / T$		$\{i\}$	$\{\$, \}$
$T \rightarrow i$		$\{i\}$	$\{+, \$\}$
	$+$	i	$\$$
F		$E \rightarrow T+E$ $E \rightarrow T$	
T		$T \rightarrow i$	

$\rightarrow$ not LL(1).

Check for LR(0) and SLR(1)

canonical collection of LR(0) items



→ State I_2 contains both reduce move and shift move, ^{means} SR conflict. Therefore this grammar is not LR(0).

✓ → But this grammar is SLR(1).

LR(0) table

	i	+	\$	E	T
0	S_3			1	2
1			accept		
2		R_2/S_4			
3	r_3	r_3	r_3		
4	S_3			5	2
5	r_1	r_1	r_1		
6					

→ SR conflict

SLR(1) table

	i	+	\$	E	T
0	S_3			1	2
1			accept		
2		S_4	r_2		
3		r_3	r_3		
4	S_3			5	2
5			r_1		

$\text{follow}(E) = \{ \$ \}$

" $\text{follow}(T) = \{ +, \$ \}$

→ no SR conflict

- ① Check given grammar are suitable for
- ✓ ~~LL(1)~~
 - ✓ ~~LR(0)~~
 - ✓ ~~SLR(1)~~

$S \rightarrow dA$
$\quad \quad \quad / aB$
$A \rightarrow bA/c$
$B \rightarrow bB/c$

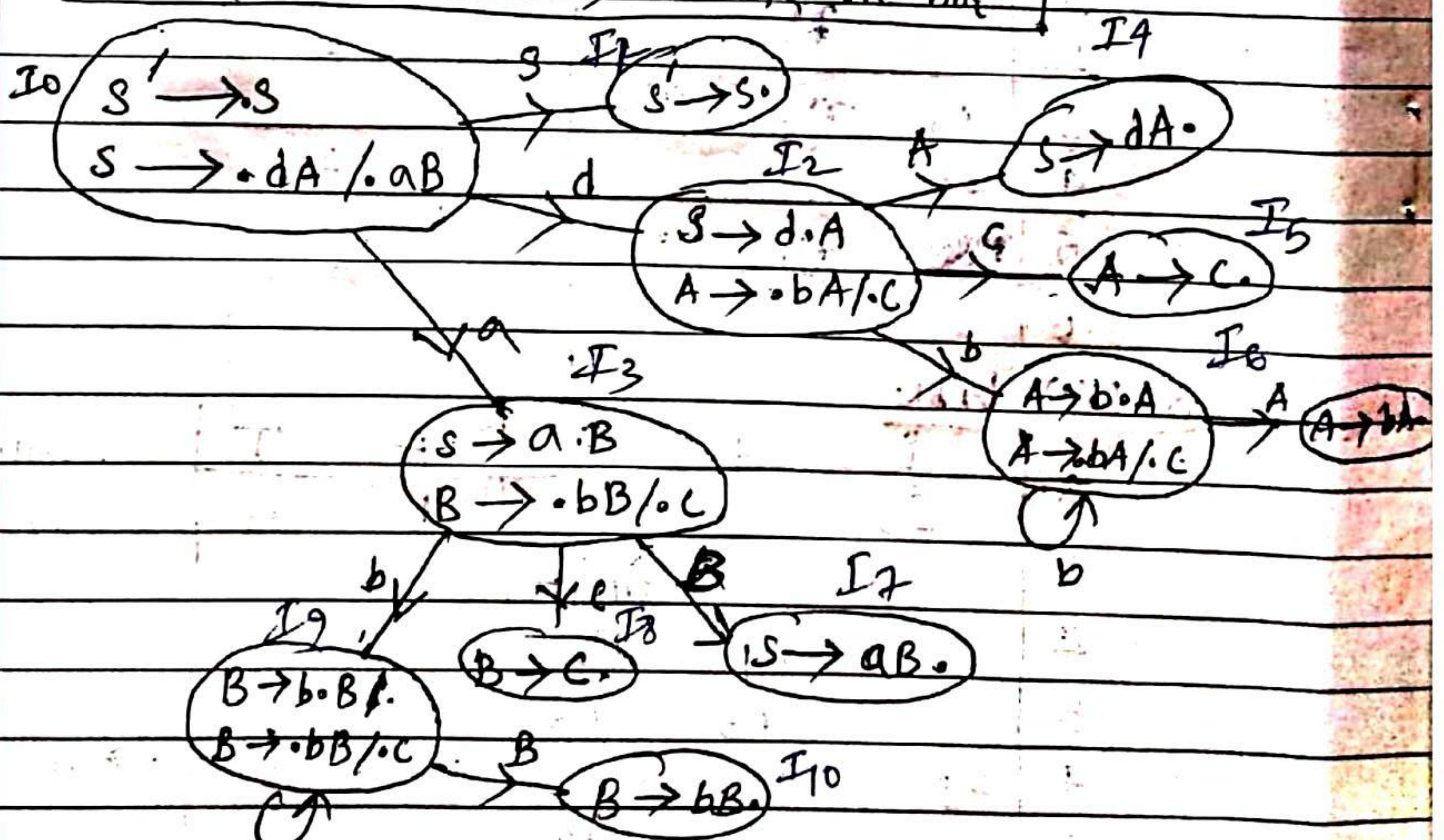
$\Rightarrow$ check for LL(1)

	first	follow
$S \rightarrow dA/aB$	$\{d, a\}$	$\{\$, \}$
$A \rightarrow bA/c$	$\{b, c\}$	$\{\$, \}$
$B \rightarrow bB/c$	$\{b, c\}$	$\{\$, \}$

	a	b	c	$\$$
S	$S \rightarrow aB$			$S \rightarrow dA$
A		$A \rightarrow bA$	$A \rightarrow c$	
B		$B \rightarrow bB$	$B \rightarrow c$	

$\rightarrow$ ~~LL(1)~~ given grammar is LL(1).

Check whether LR(0) & SLR are not



(canonical collection of LR(0) items)

→ Each final state doesn't have any SR and RR conflict, so grammar is LR(0).

→ If a grammar is suitable for LR(0) then it is also suitable for SLR(0).

⑥

⑤ Check given grammar are suitable for LL(1), LR(0), SLR(1) parsers.

	①	②	(first)
$E \rightarrow$	$E + T$	T	$\{a, b\}$
$T \rightarrow$	TF	F	$\{a, b\}$
$F \rightarrow$	$F * a$	b	$\{a, b\}$

⇒ check for LL(1) parsers

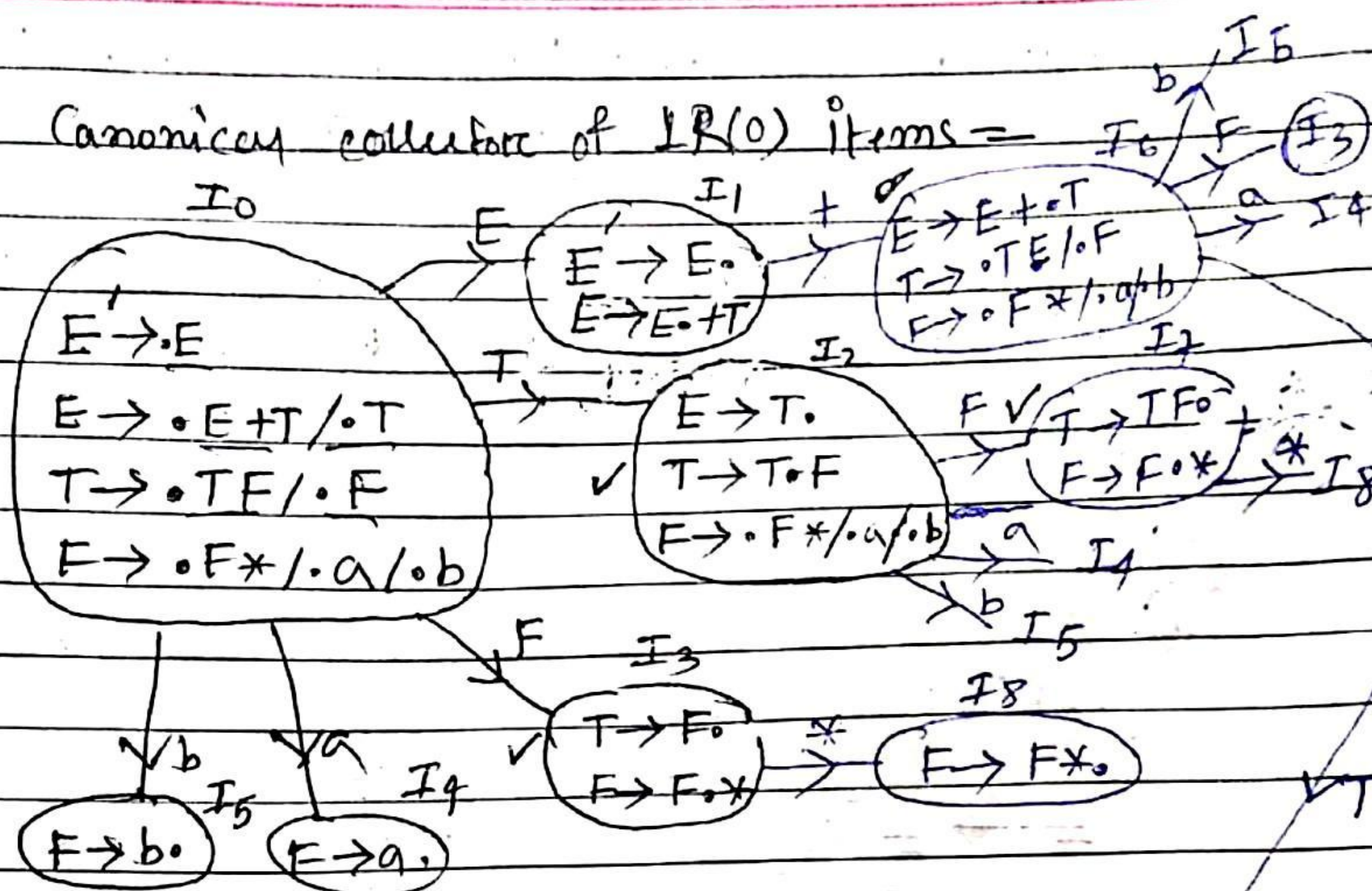
	a	b	\$
E	$E \rightarrow E + T$ $E \rightarrow T$	$E \rightarrow E + T$ $E \rightarrow T$	
T	$T \rightarrow TF$ $T \rightarrow F$	$T \rightarrow TF$ $T \rightarrow F$	
F	$F \rightarrow F * a$ $F \rightarrow b$	$F \rightarrow F * a$ $F \rightarrow b$	

→ left recursive grammar can't be suitable for LL(1) parsers.

check for LR(0) and SLR(1)

→ next page.

Canonical collection of LR(0) items =



→ This given grammar are not suitable for LR(0) parser. Because, from canonical collection of LR(0) items, we can see it have SR conflict.

~~LR(0) table~~ (shift and reduce)

SLR(1) table

	a	b	+	*	\$
2	S ₄	S ₅	r ₂		r ₂
3	r ₄	r ₄	r ₄	S ₈	r ₄
7	r ₃	r ₃	r ₃	S ₈	r ₃
9	S ₄	S ₅	r ₁		r ₁

Follow(E) = {+, \$}

Follow(T) = {+, \$, a, b}

Follow(F) = {*, +, a, b}

E T F
(not need to find
RR, SR conflict)

→ not SR/RR conflict in SLR(1) table so, given grammar are suitable for SLR(1) parsers.

$S \rightarrow \overset{\text{first}}{\{a, b\}}$ $\overset{\text{follow}}{\{\$, \}$
 $A \rightarrow \{ \epsilon \}$ $\{a, b\}$
 $B \rightarrow \{ \epsilon \}$ $\{b, a\}$

⑥ Given grammar are suitable or not =

$S \rightarrow AaAb / BbBa \{a, b\}$ ✓ LL(1)

$A \rightarrow \epsilon$ ③ $\{ \epsilon \}$ ② ✗ LR(0)

$B \rightarrow \epsilon$ ④ $\{ \epsilon \}$ ✗ SLR(1)

LL(1) :

⇒

	a	b	\$
S	$S \rightarrow AaAb$	$S \rightarrow BbBa$	
A	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	
B	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$	

Note:

$A \rightarrow \cdot \epsilon$
 $A \rightarrow \epsilon \cdot$ } $A \rightarrow \cdot$ Same

→ each cell have only one production, so given grammar are accepted by LL(1) parsers.

Check for LR(0) and SLR(1) :

$S \rightarrow \cdot S$
 $S \rightarrow \cdot AaAb / \cdot BbBa$
 $A \rightarrow \cdot$
 $B \rightarrow \cdot$

→ This grammar not LR(0), because one single state contain two reduce move.

SLR(1) parsing table

	a	b	\$
0	r_3/r_4	r_3/r_4	

→ Reduce-Reduce conflict, so this

this grammar are not accepted for SLR(1) parser.

⑦ Check whether a grammar is ^{given} $LL(1)$, $LR(0)$, $SLR(1)$. ^{once}

$$S \rightarrow AS \text{ ①}$$

$$/b \text{ ②}$$

$$A \rightarrow SA \text{ ③}$$

$$/A \text{ ④}$$

$$\{a, b\}$$

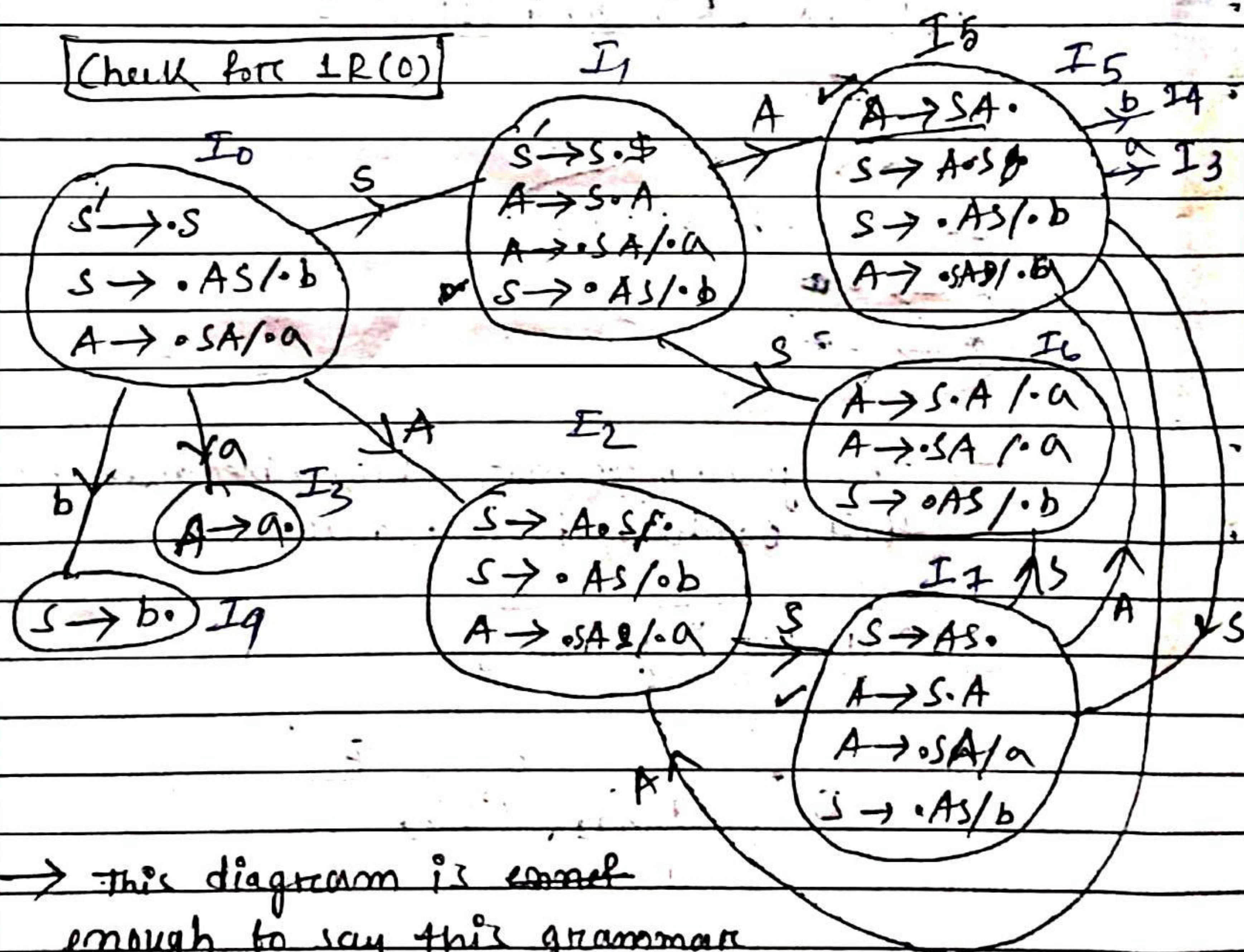
$$\{a, b\}$$

⇒ Check for $LL(1)$

	a	b	.	\$
S	$S \rightarrow AS$	$S \rightarrow AS$ $S \rightarrow b$		
A	$A \rightarrow SA$ $A \rightarrow a$	$S \rightarrow SA$		

→ there more than two production in one cell, so, this grammar can't be $LL(1)$ grammar.

Check for $LR(0)$



→ this diagram is enough to say this grammar is not $LR(0)$, because (SR conflict).

[Check for SLR(1)]

SLR(1) parsing table :-

	a	b	\$
5	$\overset{\circ}{r} 3/s_3$	$\overset{\circ}{r} 3/s_4$	

follow(A) = {a, b}

) (5) = {\$, a, b}

→ LR conflict.

→ grammar not accepted by SLR(1) parser.

Given grammar are ambiguous, so that this grammar are not accepted by LL(1), LR(0) and SLR(1) parser.

⑧ Check given grammar are ^XLL(1), ^XLR(0) and ^XSLR(1) or not.

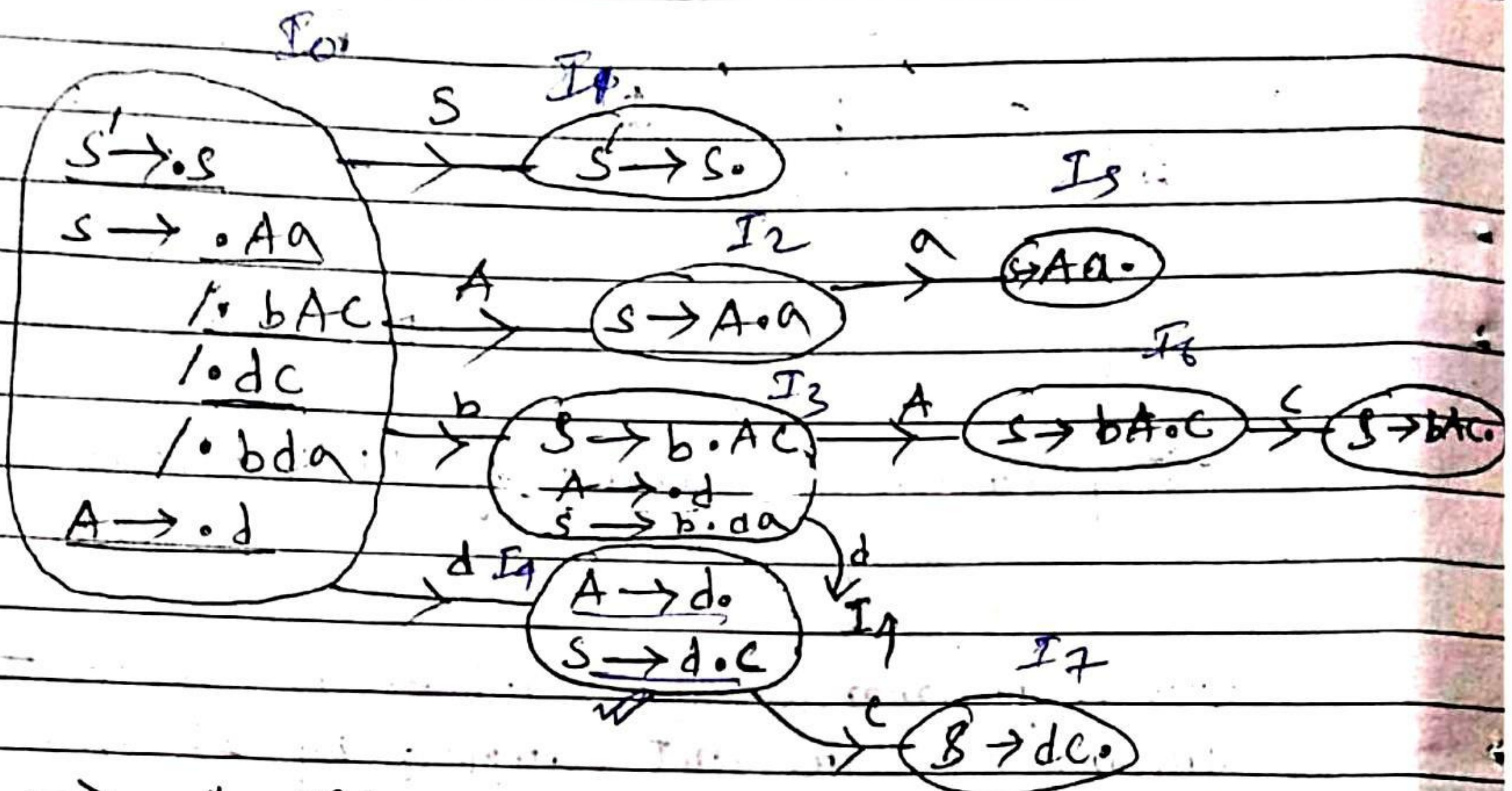
	first
$S \rightarrow Aa$ ①	{a, b}
$\quad \quad \quad / bAC$ ②	
$\quad \quad \quad / dC$ ③	
$\quad \quad \quad / bda$ ④	
$A \rightarrow d$ ⑤	{d}

⇒ [Check for LL(1) parser]

	a	b	c	d	\$
S		$S \rightarrow bAC$ $S \rightarrow bda$		$S \rightarrow Aa$ $S \rightarrow dC$	
A				$A \rightarrow d$	

→ not LL(1).

[Check for LR(0)] =



→ not LR(0).
(because LR conflict).

follow(A) = {a, c}

[Check for SLR(1)] =

SLR(1) table =

	a	b	c	d	\$
I0	r5		r5/s7		

→ LR conflict

→ not accepted by SLR(1) parser.

• LR(1) and LALR(1) parsers :

CLR(1) LALR(1)

LR(1) item = LR(0) item + lookahead.

• canonical collection of LR(1) items -

Given grammar,
 $S \rightarrow AA$
 $A \rightarrow aA/b$

(lookahead)

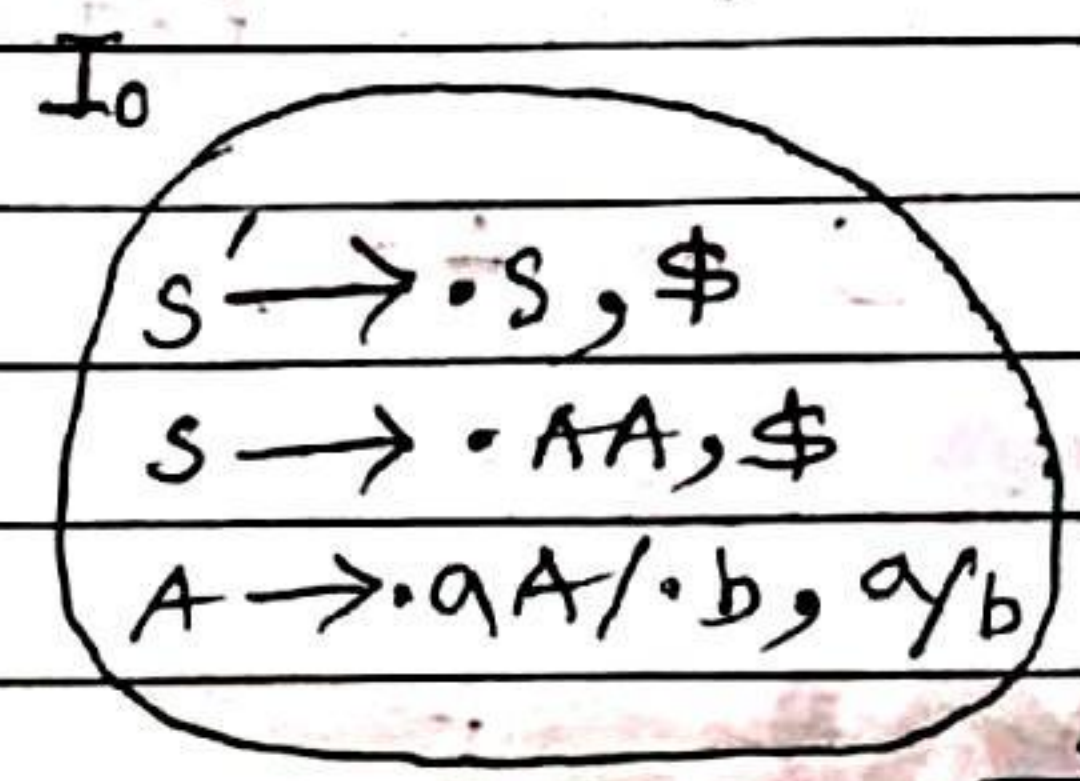
$A \rightarrow a.Bb, a/b$

$B \rightarrow a.y, c/a$

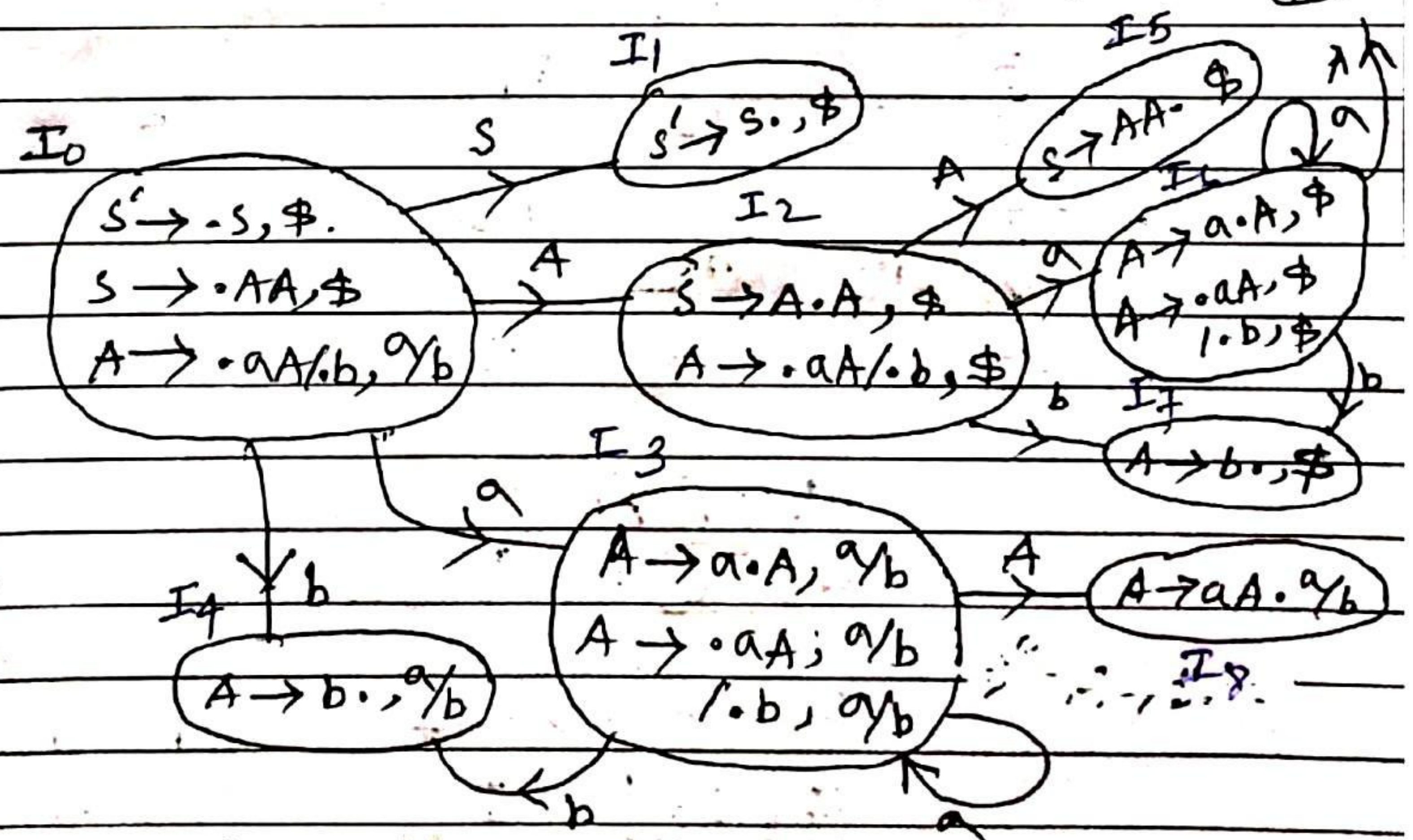
first of B

$A \rightarrow a.B, a/b$

$B \rightarrow a.y, a/b$



→ after transition of any symbol
 Lookahead doesn't change.



→ state may increase in LR(1) compare to LR(0) parser.

- Number of state increases because of Lookahead.
- Number of Reduce move decrease because of Lookahead.
- In diagram I_3 and I_6 both have same LR(0) items but different Lookahead.
- also I_4 and I_7 both have same LR(0) items but diff Lookahead.
- I_8 and I_9 " " " "

• CLR(1) parsing table (for previous example)

	a	b	\$	S	A
0	S_3	S_4			2
1					
2	S_6	S_7			5
3	S_3	S_7			8
4	r_3	r_3			
5			r_1		
6	S_6	S_7			9
7			r_3		
8	r_2	r_2			
9			r_2		

(Merging)

$$[I_3, I_6] \Rightarrow I_{36}$$

$$[I_4, I_7] \Rightarrow I_{47}$$

$$[I_8, I_9] \Rightarrow I_{89}$$

$$S \rightarrow AA \text{ (1), } \$$$

$$A \rightarrow aA \text{ (2)}$$

$$A \rightarrow b \text{ (3), } a/b$$

(reduce move according to lookahead)

$$\checkmark \text{ no. of state LR(0) = SLR(1) } \leq \text{CLR(1)} \\ = \text{LALR(1)}$$

• LALR(1) parsing table

→ LALR(1) table made from CLR(1) table, by merging '6' state into '3' states.

	a	b	\$	S	A
0	S ₃₆	S ₄₇			2
1					
2	S ₃₆	S ₄₇			5
36	S ₃₆	S ₄₇			89
47	r ₃	r ₃			
5			r ₁		
36	S ₃₆	S ₄₇			89
47			r ₃		
89	r ₂	r ₂			
89			r ₂		

(compress similar states)

New table

	a	b	\$	S	A
0	S ₃₆	S ₄₇			2
1					
2	S ₃₆	S ₄₇			5
36	S ₃₆	S ₄₇			89
47	r ₃	r ₃	r ₃		
5			r ₁		
89	r ₂	r ₂	r ₂		

(no of state, LR(0) = SLR(1) = LALR(1) = < CLR(1) >)

(7) (7) (7) (10)

- SR conflict and RR conflict =

LR(1) items: $\begin{matrix} A \rightarrow \alpha \cdot a \beta, \epsilon/d \\ B \rightarrow \gamma \cdot, \epsilon/y\$ \end{matrix} \xrightarrow{\text{a}} \boxed{\text{SR conflict}}$

(single state contain shift and reduce move both)

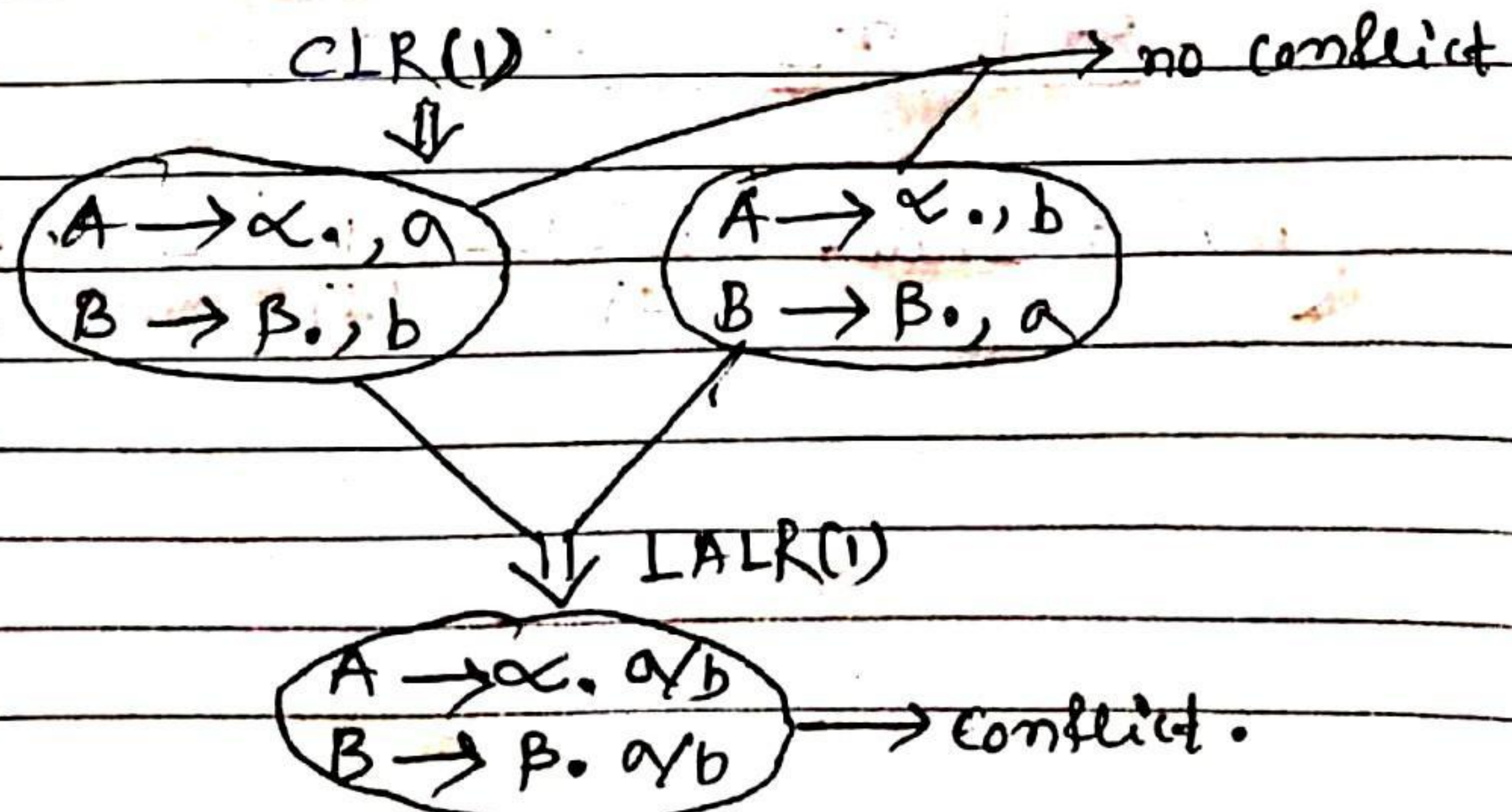
$\begin{matrix} A \rightarrow \alpha \cdot, a \\ B \rightarrow \alpha \cdot, a \end{matrix} \xrightarrow{\text{a}} \boxed{\text{RR conflict}}$

(single state contain Reduce-Reduce move).

→ If '1' grammar are not accepted by CLR(1) parser, that can't be accepted by LALR(1) parser.

→ When one grammar are accepted by CLR(1) parser, that grammar may or may not be accepted by LALR(1) parser.

ex =

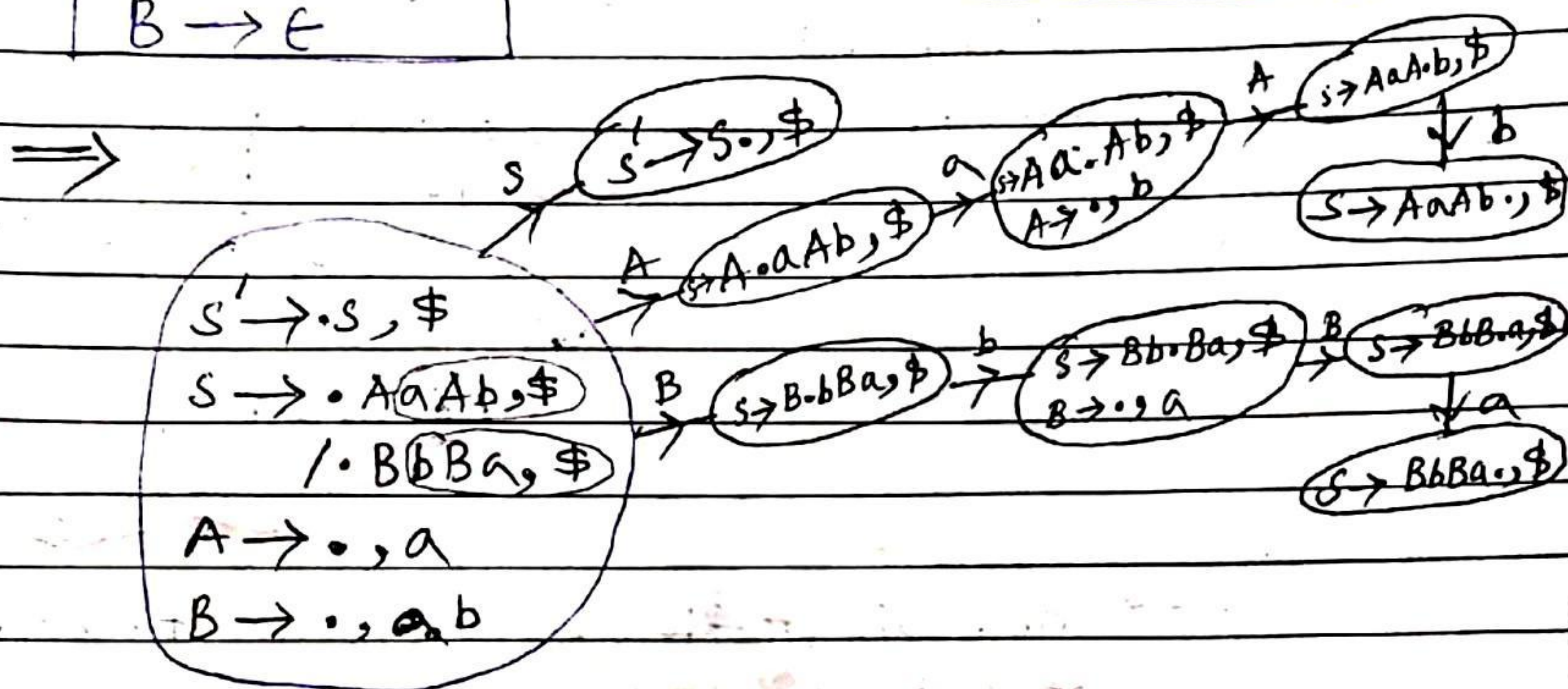


Example =

(1) Check given grammar are accepted by - ~~(i) CLR(1)~~
(ii) LALR(1).

$S \rightarrow$	$AaAb$ $/ BbBa$
$A \rightarrow$	ϵ
$B \rightarrow$	ϵ

$A \rightarrow \cdot \epsilon \rightarrow \epsilon \cdot \rightarrow \cdot$



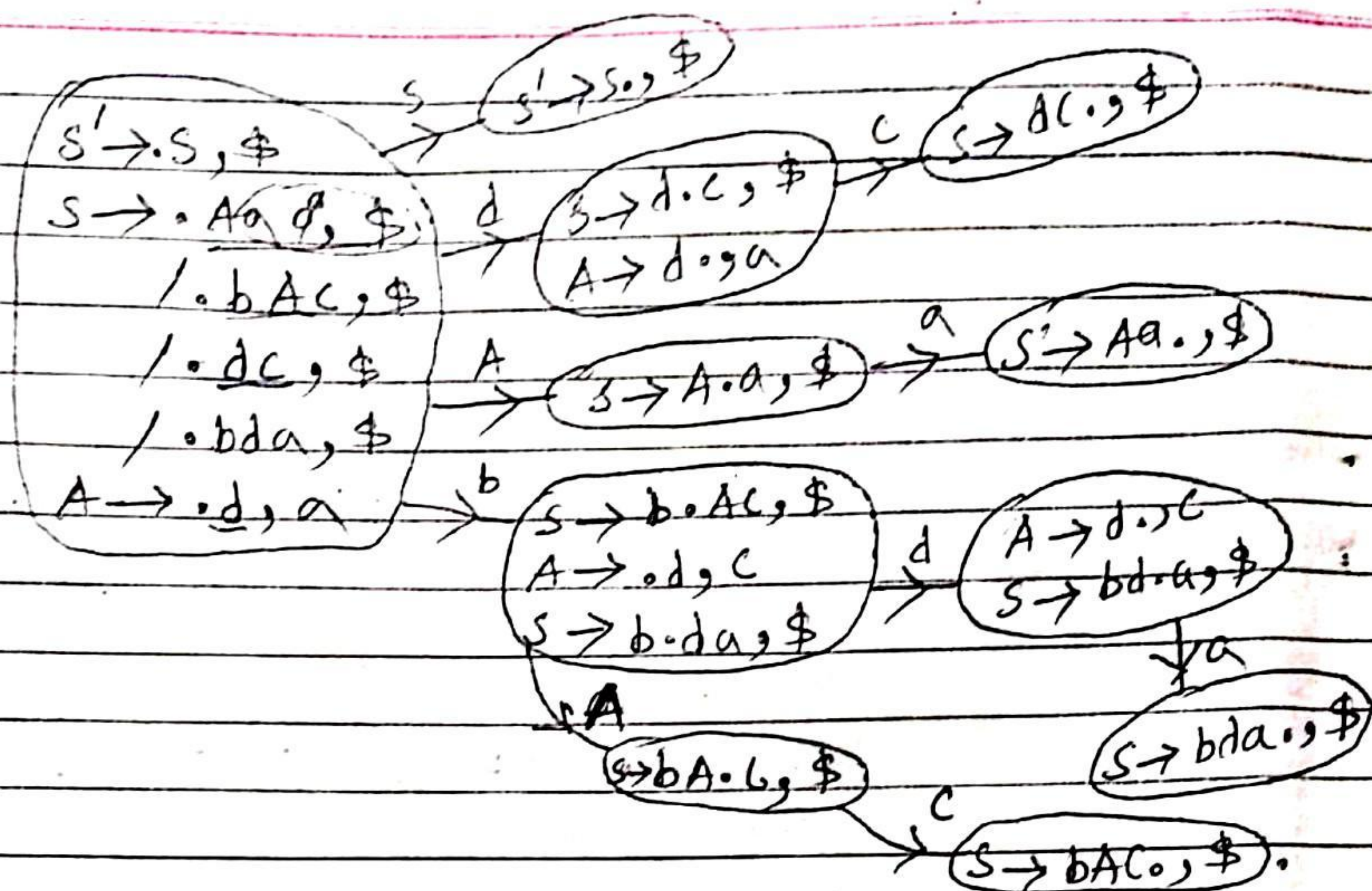
Canonical Collection of LR(1) items, no conflict occurs. So, given grammar accepted by CLR(1) parser. And state merging not possible so that grammar also accepted by LALR(1) parser.

(2) Given grammar are accepted by - ~~✓~~ LR(1) parser.
~~✓~~ LALR(1) parser.

$S \rightarrow$	Aa $/ bAc$ $/ dc$ $/ bda$
$A \rightarrow$	d

$\Rightarrow$ Canonical collection of LR(1) items =

$\rightarrow$ next page.



→ no, SR ~~or~~ PR and RR conflict. And number of state can't be reduced by (merge)

→ so, given grammar are accepted by CLR(1) or LALR(1) both, parser.

③ Check given grammar are accepted by — CLR(1) ~~and~~, LALR(1) parser, LL(1), LR(0), SLR(1).

$S \rightarrow Aa$ $\{a, b\}$
 $\quad \quad \quad / bAa$
 $\quad \quad \quad / Bc$
 $\quad \quad \quad / bBa$

$A \rightarrow d$ $\{a\}$
 $B \rightarrow d$ $\{a\}$

⇒ LL(1):

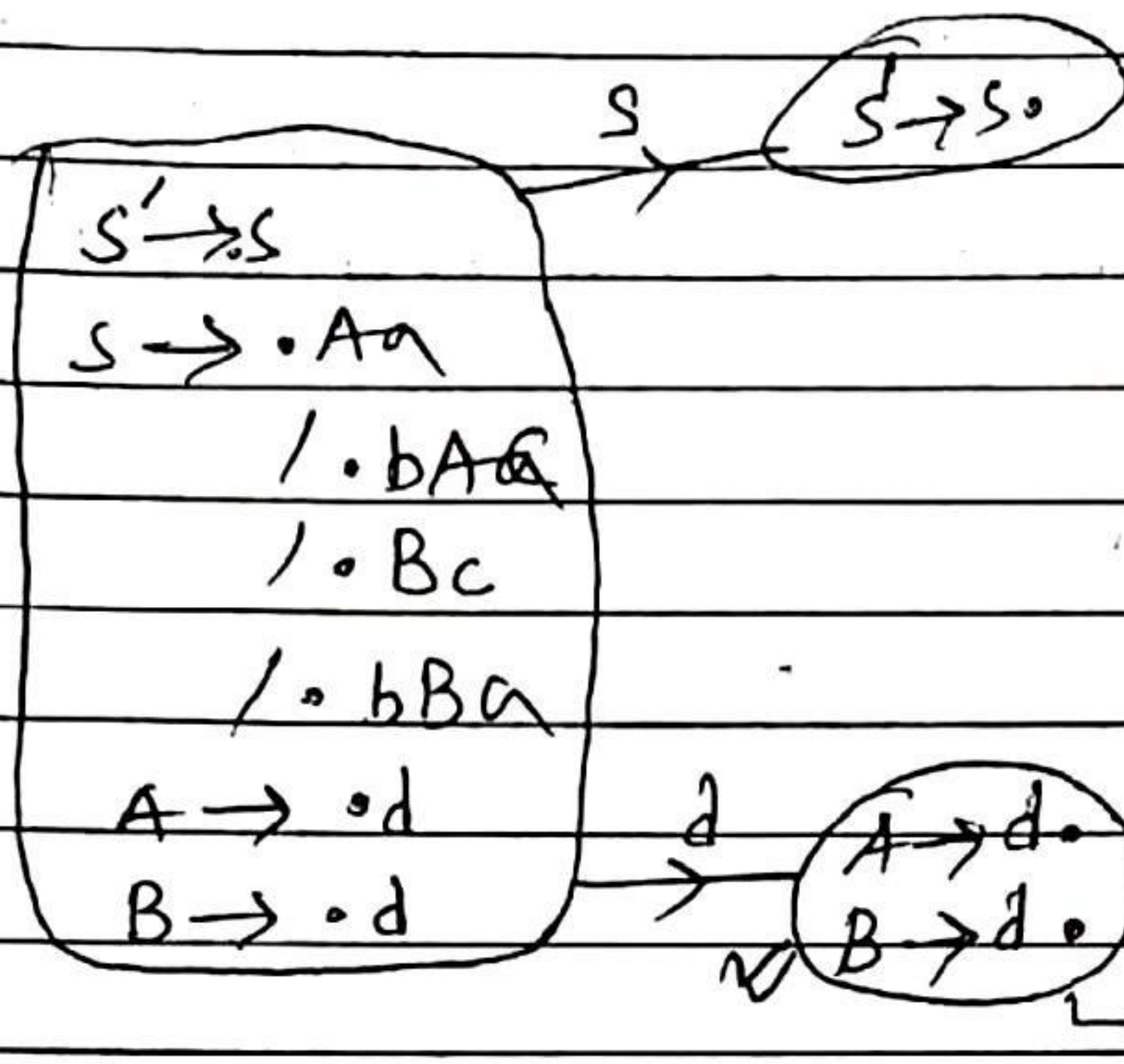
	a	b	c	d	\$
S		$S \rightarrow bAa$ $S \rightarrow bBa$		$S \rightarrow Aa$ $S \rightarrow Bc$	
A				$A \rightarrow d$	
B				$B \rightarrow d$	

— not accepted by LL(1) parser.

Check for LR(0) = SLR(1) also.

Canonical collection of LR(0) items:

(not required to draw full diagram) to prove.



Follow(A) = {a, c}

Follow(B) = {a, c}

RR conflict.

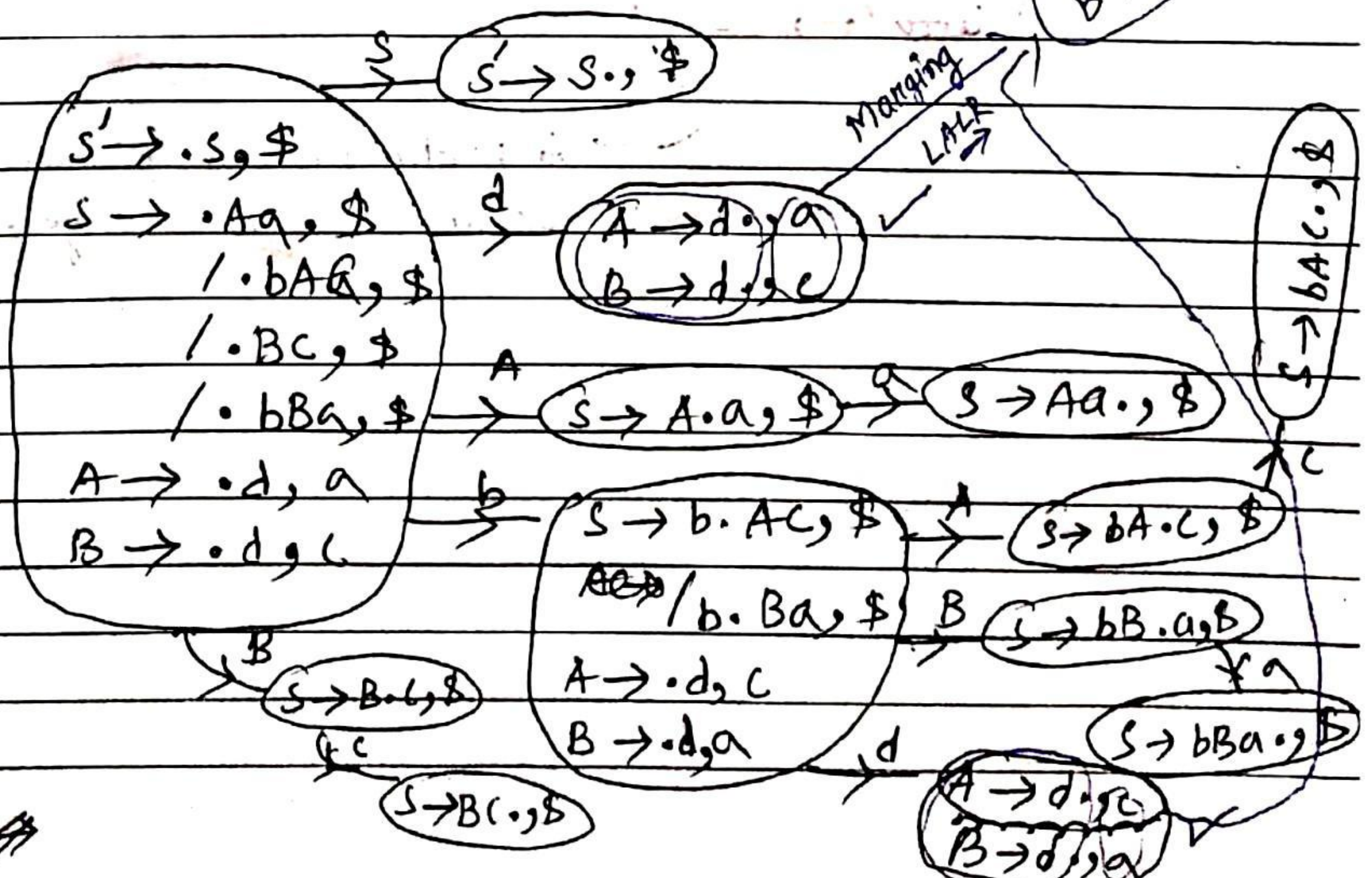
Follow

→ not accepted by LR(0) (because RR conflict)

→ also not accepted by SLR(1) (RR conflict)

Check for CLR(1) and ALR(1)

Canonical collection of LR(1) items -



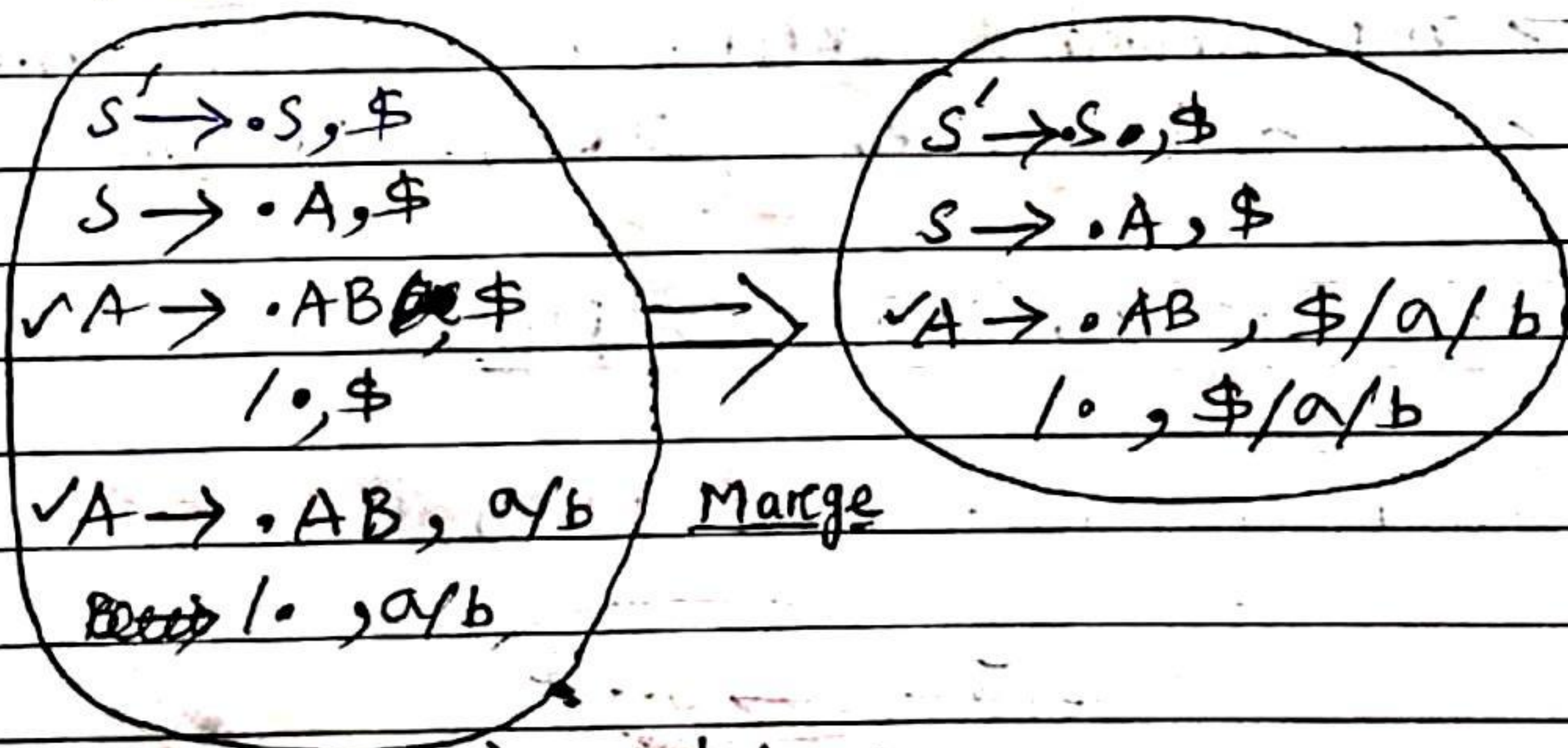
→ given grammar accepted by LR(1) parsers.
(No RR and SR conflict)

→ given grammar are not accepted by LALR(1)
(because after merging two state in one state
then new state contain ~~two~~ ~~two~~ RR conflict.

④ Example = ①

$S \rightarrow A$
$A \rightarrow AB/\epsilon$
$B \rightarrow aB/b$

Closure

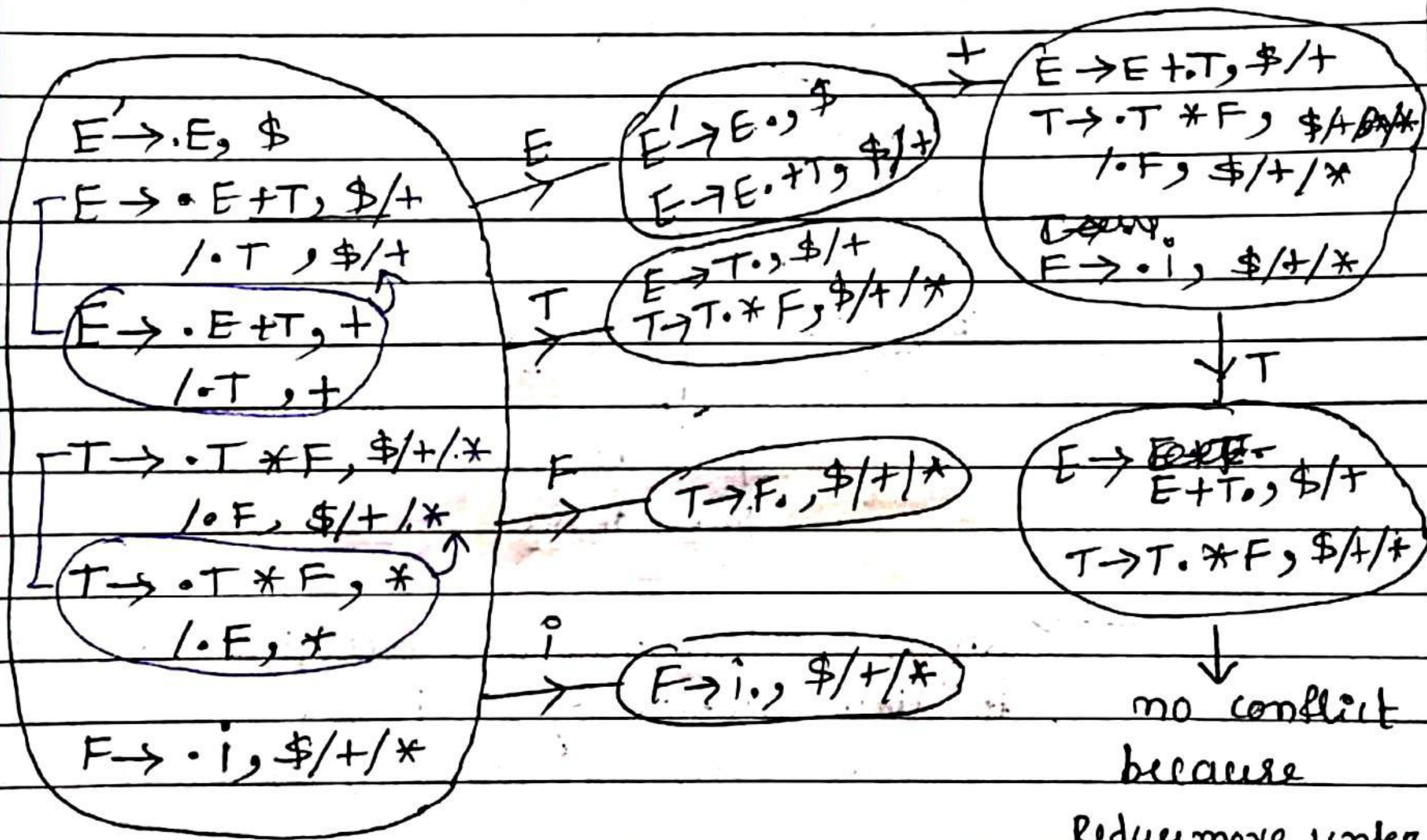


→ 'A' written two time because there
lookahead can change.

example - (2)

$$\begin{array}{l} E \rightarrow E+T \\ \quad /T \\ T \rightarrow T * F \\ \quad /F \\ F \rightarrow i \end{array}$$

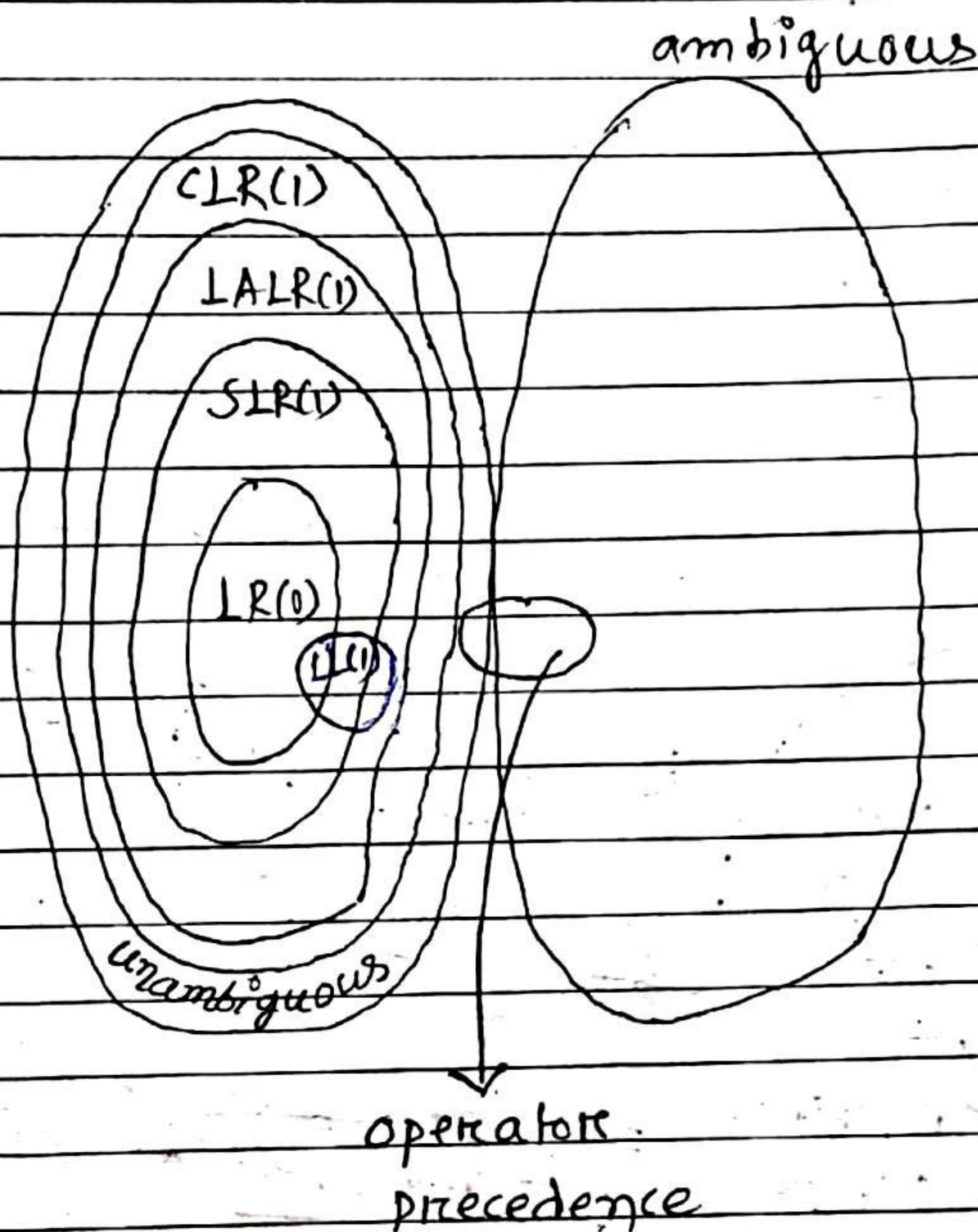
canonical collection of LR(1) items =



no conflict
because

Reduce move under
 $\$, +$ and
shift move
under $*$

• Comparison of all parsers =



→ when a grammar are accepted by $LL(1)$, then it also accepted by $LALR(1)$.
(because $LL(1)$ are completely under of $LALR(1)$).

• Some questions =

Q-1 consider $SLR(1)$ and $LALR(1)$ table for a CFG -

X (a) Goto of both tables may be different.

✓ (b) shift entries are identical in both tables.

✓ (c) Reduce entries in table may be different.

✓ (d) error entries in tables may be different.
(Blank entries)

[Q-2] n_1, n_2 and n_3 are number of state of SLR(1), LALR(1) and CLR(1) ^{table} respectively. What is the relationship between n_1, n_2 and n_3 .

$$\rightarrow n_1 = \text{SLR(1)}$$

$$n_2 = \text{LALR(1)}$$

$$n_3 = \text{CLR(1)}$$

relation -

$$n_1 = n_2 \leq n_3$$

[Q-3] $S \rightarrow CC$

$C \rightarrow cC/d$

$\rightarrow$ this grammar are accepted by all grammars.

☒ a) LL(1)

☐ b) SLR(1) but not LL(1)

☐ c) LALR(1) but not SLR(1)

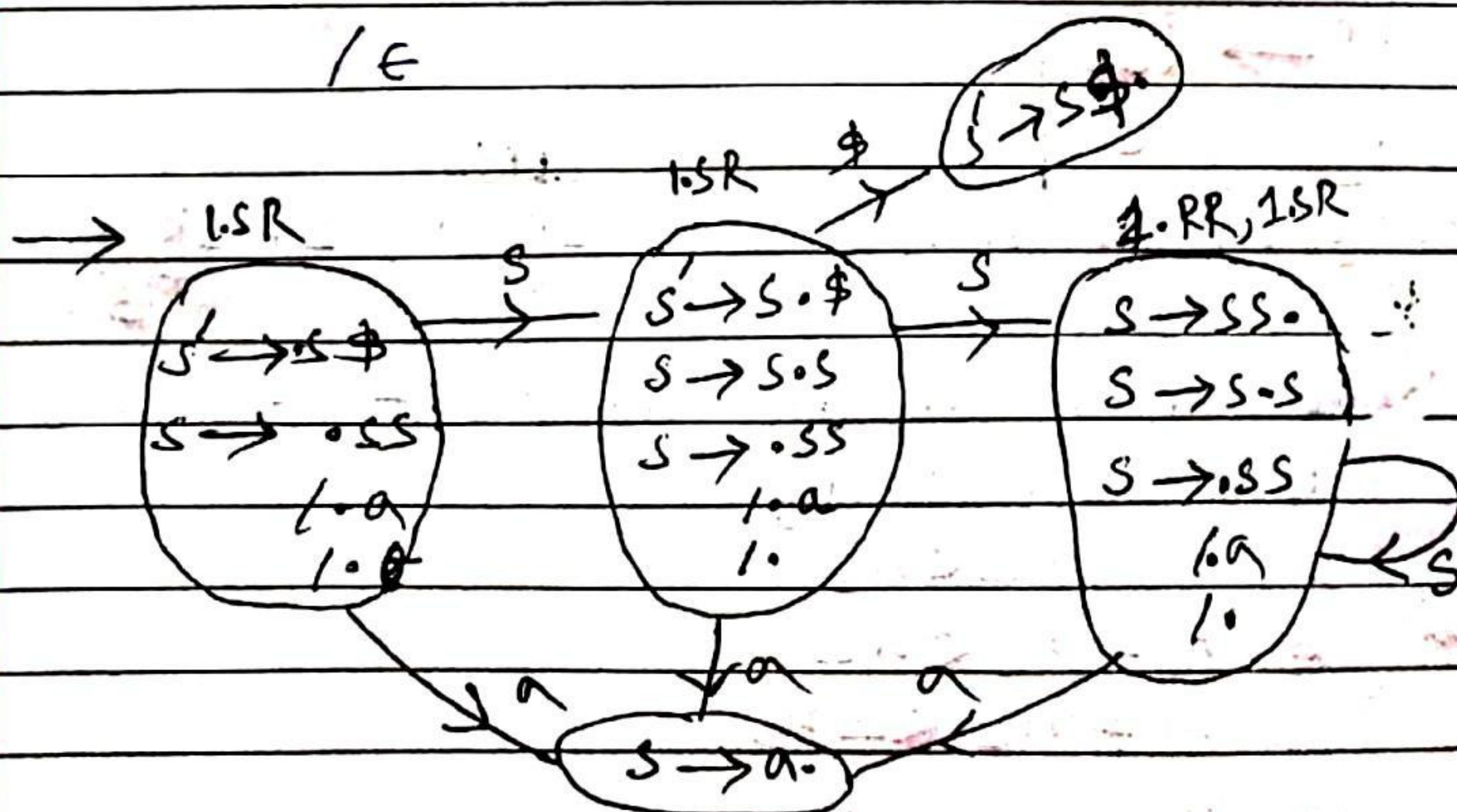
☐ d) CLR(1) but not LALR(1).

[Q-4] Find the numbers of SR and RR conflicts in dfa with LR(0) items =

$S \rightarrow SS$

/a

/ε



$\rightarrow$ hence 3-SR move and 1-RR move.

[8-5] given-

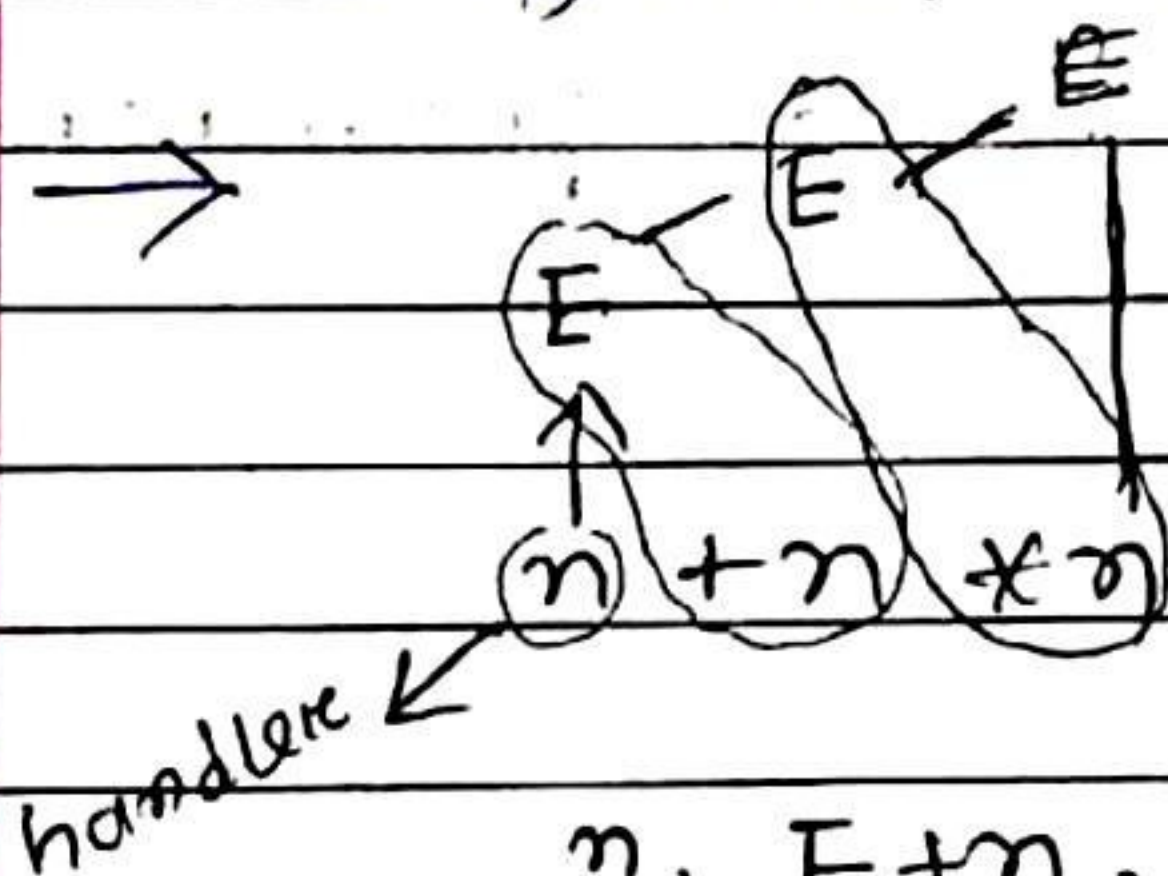
$$E \rightarrow E + \eta$$

$$/ E * \eta$$

$$/ \eta$$

for the string $\eta + \eta * \eta$, the handles in right sentential form of reduction are -

- $\eta, E + \eta, E + \eta * \eta$
- $\eta, E + \eta, E + E * \eta$
- $\eta, \eta + \eta, \eta + \eta * \eta$
- $\eta, E + \eta, E * \eta$



or

~~or~~

$$E \Rightarrow E * \eta$$

$$\Rightarrow E + \eta * \eta$$

$$\Rightarrow \eta + \eta * \eta$$

[8-6] given grammar =

$$S \rightarrow (S_1)$$

$$/ a$$

relation between $SLR(1)$, $LR(1)$, $LALR(1)$

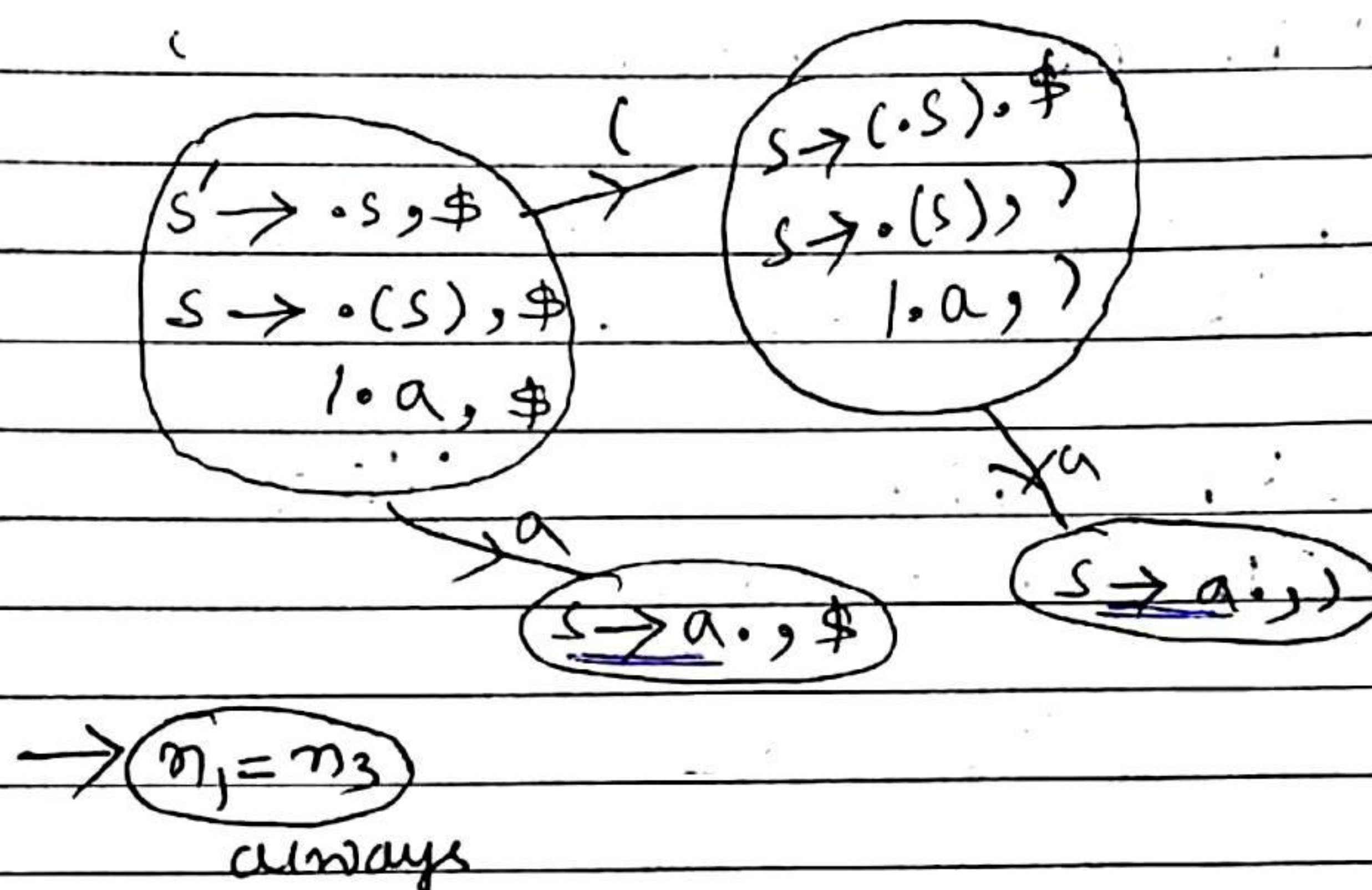
$n_1 \quad n_2 \quad n_3$

here n_1, n_2, n_3 are no. of state of $SLR(1), LR(1), LALR(1)$ respectively.

- $n_1 < n_2 < n_3$
- $n_1 = n_3 < n_2$
- $n_1 = n_2 = n_3$
- $n_1 \geq n_3 \geq n_2$

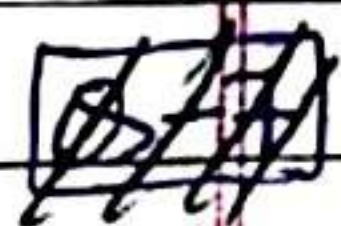
$$\Rightarrow (n_1 = n_3 \leq n_2)$$

canonical collection for LR(0) items -



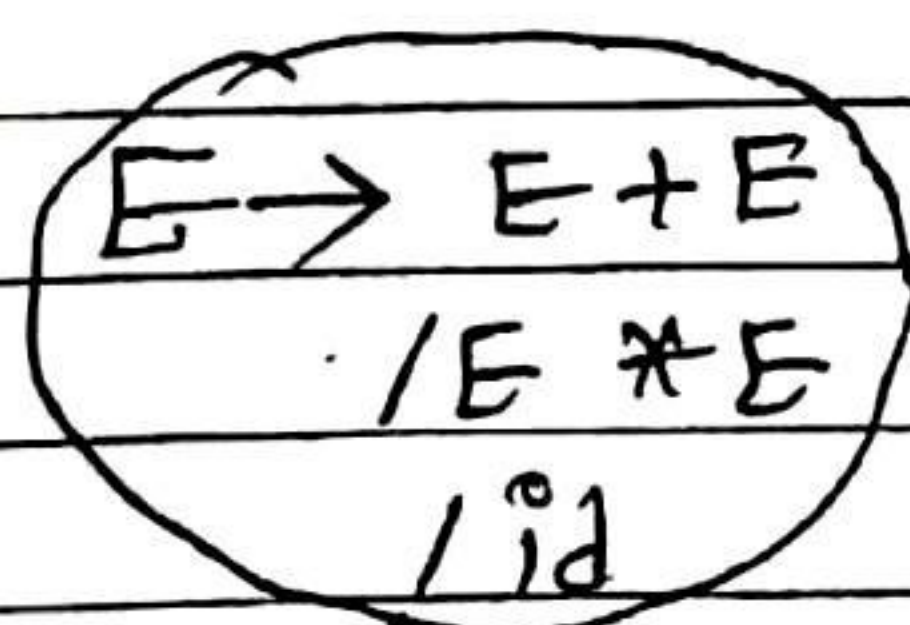
→ ^{of LR(1)} no. of state ^{is} more compare to LR(0).

$$n_1 = n_3 < n_2$$



• Ambiguous and unambiguous =

Ambiguous

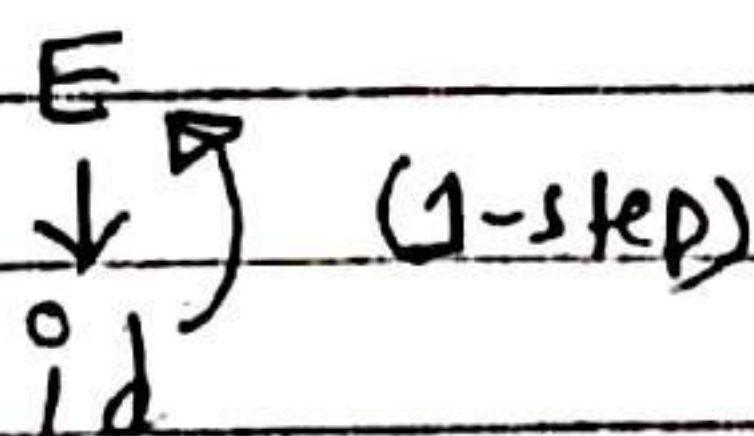


Adv:

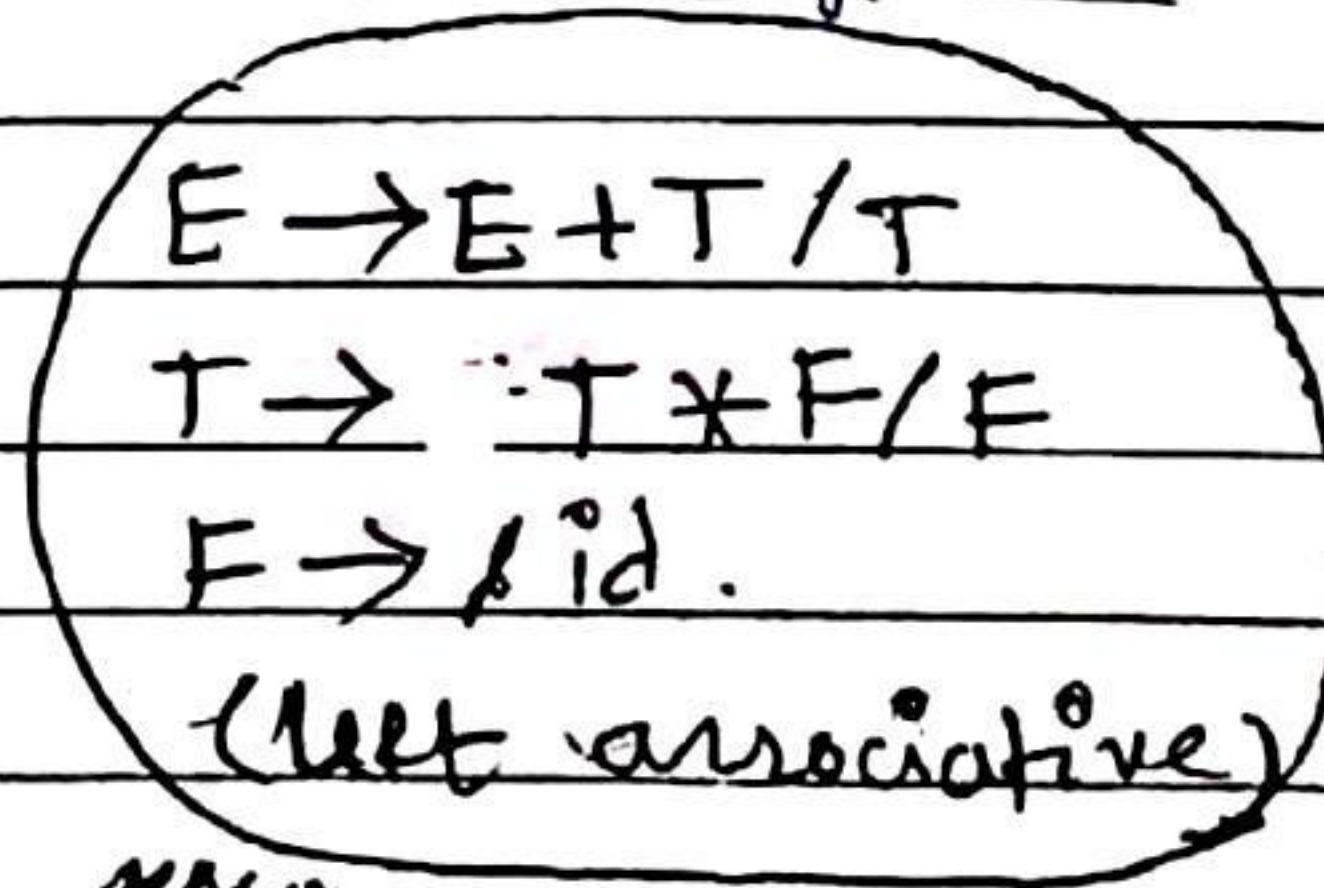
(i) no. of production less.

(ii) natural and easy to understand.

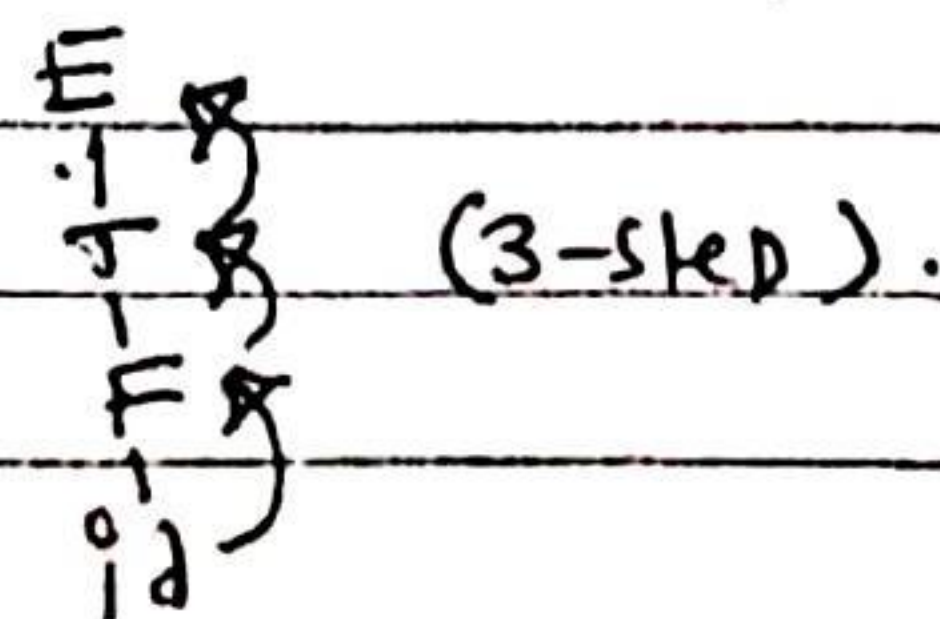
(iii) we can change associativity and precedence.



Unambiguous



Adv:



→ ambiguous grammar not used in parsing
expect operator precedence because of conflict.
will be occur more.

→ ambiguous can used, if you can solve the
conflicts meaningfully.

$\boxed{SR} \rightarrow \text{shift}$
 $\boxed{RR} \rightarrow \text{first Reduce}$

$\boxed{YACC} = \text{use table of}$
 LALR(1)

r_1/r_2